

Zona Web

Complementos a temario aula

None

None

Table of contents

1. Inicio	3
1.1 Bloque HTML	3
1.2 Bloque CSS	3
1.3 Bloque JS	3
2. 01. HTML	4
2.1 Introducción	4
2.2 Ampliación Etiquetas Semánticas de estructura	5
2.3 Cap 04 Ampliación Enlaces	7
2.4 Varios Ampliación	9
2.5 imágenes de prueba	11
2.6 Cap. 08	14
2.7 Errores frecuentes Bloque HTML	19
2.8 Varios	21
3. 02. CSS	22
3.1 Introducción	22
3.2 Ampliación Selectores Uniwebsidad	23
3.3 Colores	26
3.4 Box model	27
3.5 CSS y texto	29
3.6 Layout HTML con CSS grid	34
3.7 Diseño Adaptativo (RWD)	38
3.8 Errores en CSS	42
3.9 Amplia	43
4. 03. JavaScript	56
4.1 Introducción JavaScript 01	56
4.2 Introducción JavaScript 02	61
4.3 Introducción JavaScript 03 - DOM	75
4.4 JavaScript Abreviado	84
4.5 Linter JavaScript: eslint 24/25	86

1. Inicio

1.1 Bloque HTML

Compuesto por

- Los apuntes de Uniwebsidad disponibles en:
 - a. En este enlace: <https://web.archive.org/web/20241207123740/https://uniwebsidad.com/libros/xhtml/>
 - b. Y aquí tiene una [copia en PDF](#)
- Y los complementos que tienes en el índice, organizados por capítulos.

1.2 Bloque CSS

Compuesto por:

- Los apuntes de Uniwebsidad disponibles
 - a. en este enlace <https://web.archive.org/web/20241207111819/https://uniwebsidad.com/libros/css>
 - b. Y aquí una [copia en PDF](#)
- Y los complementos que tienes en el índice, organizados por capítulos.

1.3 Bloque JS

2. 01. HTML

2.1 Introducción

Tiene un [esquema de este bloque en este enlace](#)

Este bloque está compuesto por:

- Los apuntes de Uniwebsidad disponibles en:
- En este enlace: <https://web.archive.org/web/20241207123740/https://uniwebsidad.com/libros/xhtml/>
- Y aquí una [copia en PDF](#)
- Y los complementos que tienes en el índice, organizados por capítulos.

2.2 Ampliación Etiquetas Semánticas de estructura

2.2.1 1. Etiqueta semánticas

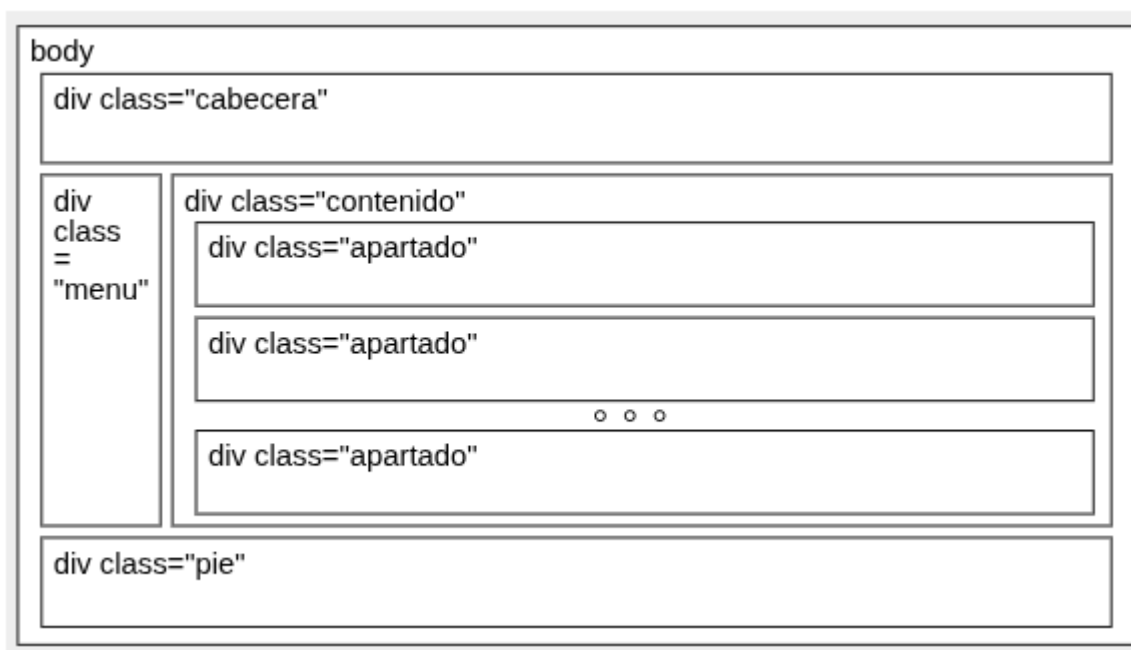
La referencia principal de consulta en este caso es:

- [Secciones HTML](#) en la web de MCLIBRE - MATERIAL CURRICULAR LIBRE

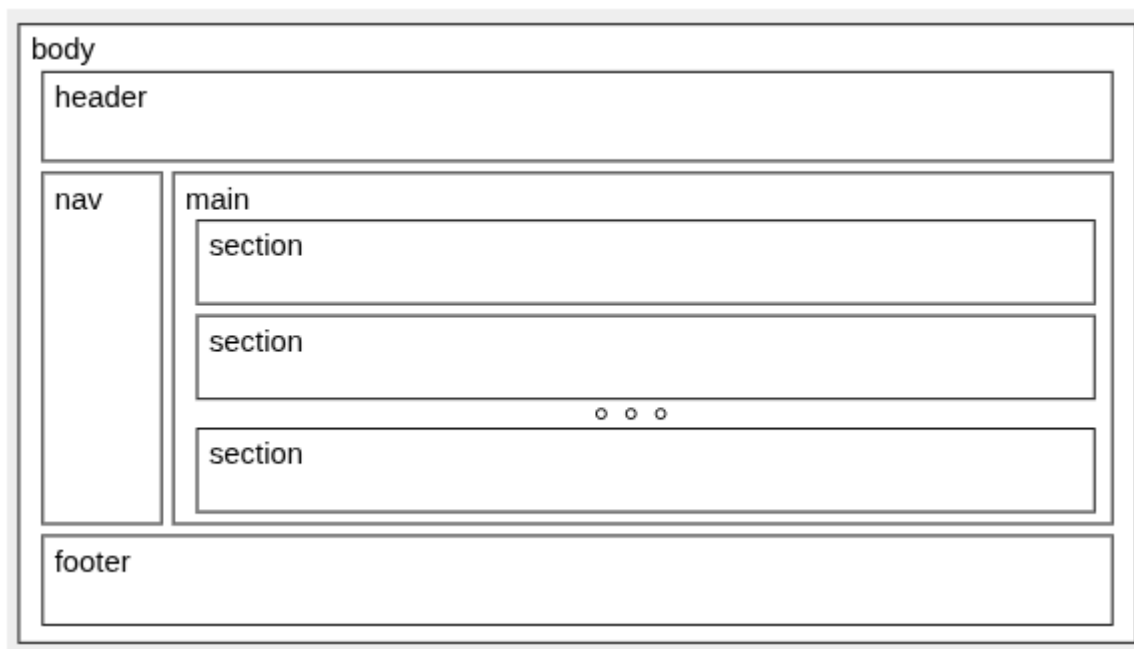
Los elementos que nos interesa conocer sobre todo son:

- **nav** : está pensada para agrupar los enlaces a otras páginas del sitio web o a apartados de la propia página.
- **header** : está pensada para agrupar el contenido inicial de una página web: el logotipo, el título principal de la página o texto de introducción.
- **footer** : está pensada para contener información general sobre el documento, información que se suele poner al final del documento: autor, direcciones de contacto, licencia o condiciones de uso, enlaces a otros documentos relacionados, etc.
- **aside** : está pensada para agrupar contenido secundario y tangencial al contenido al que acompaña por ejemplo, un bloque de anuncios.
- **section** : para agrupar apartados del contenido relacionados. Normalmente empiezan con un título que hace referencia al tema tratado en la sección.
- **article** : está pensada para agrupar el contenido de la página que forma una unidad en sí misma desde el punto de vista temático. Un artículo debería poder publicarse y leerse como documento independiente,

Observe un ejemplo del antes



y el después con el uso de estos elementos:



2.2.2.2. Código para hacer pruebas

- [Ejemplo uso elementos semánticos](#)
- [Outline](#) para usar con plugin HeadingsMap

2.2.3.3. Otras referencias a consultar

- [HTML Semantic Elements](#) de W3schools,

3.1. Para los muy curiosos

- [HTML5 section, aside, header, nav, footer elements, Not as obvious as they sound](#)
- HTML5 Doctor Element [Flowchart](#), [Sectioning mcontent elements and friendss](#)

2.3 Cap 04 Ampliación Enlaces

2.3.1 1. Usar una imagen como enlace

Basta con "envolver" la imagen con el elemento `a` :

```
<a href="http://acme.org">
  
</a>
```

2.3.2 2. Enlaces absolutos y relativos: otro ejemplo

Se añade una URL con un ejemplo adicional al de uniwebsidad.

- [Cómo usar rutas absolutas y rutas relativas](#) del blog de Hostalia

2.3.3 3. Extensiones tipo Live Server en VsCode

En general se recomienda iniciar VSCode abriendo una carpeta o directorio, y esta se convierte en la raíz de nuestro proyecto.

Al usar la extensión *Live Server* o similares se inicia un servidor web de desarrollo. Dicho servidor web usa como raíz del sitio web dicha carpeta. Esto es importante a la hora de especificar rutas relativas a la raíz. Si inicia VS Code con un fichero en vez de una carpeta, al intentar usar este tipo de complementos mostrará un error y le indicará que es necesario abrir una carpeta (*folder*).

2.3.4 4. El Atributo `download`

Tomado de *MDN*:

Este atributo indica *descargar* a los navegadores una URL en lugar de navegar hacia ella, por lo que el usuario será dirigido para salvarla como un archivo local. Si el atributo tiene un valor, éste se utilizará como nombre de archivo por defecto en el mensaje *Guardar* que se abre cuando el usuario hace clic en el enlace (sin embargo, el usuario puede cambiar el nombre antes de guardar el archivo).

Ejemplo de su uso:

```
<a href="/images/myw3schoolsimage.jpg" download>
```

2.3.5 5. El atributo `target`

Los valores posibles del atributo `target` son:

- `target="_blank"` : abre siempre el enlace en una nueva pestaña o ventana del navegador
- `target="_self"` : es la opción por defecto, muestra el contenido del enlace en la misma ventana
- `target="nombre"` : si se indica un nombre se abre el contenido en una pestaña con ese nombre. Si hay otros enlaces con ese nombre se usará dicha pestaña para mostrar el contenido.

Y si usara frames hay otras opciones, pero los framesets están ya obsoletos (deprecated).

Frames deprecated; only use `_blank`

The only currently relevant value of target is `_blank`. The other values of target were used to specify specific frames. However, frames have been deprecated in HTML5.

Default target

If no target is specified, the link will open in the current context, unless the user or browser specifies otherwise. Read more: <https://html.com/attributes/a-target/#ixzz7g581fqxs>

Aviso: si usa "_blank" como target debería leer algunas cuestiones de seguridad:

- [La vulnerabilidad target=»_blank» en WordPress](#)
- [Don't Use target="_Blank"](#)

2.3.6 6. Ejercicios

1. [Ejercicio de enlaces](#) para clase.
2. [Ejemplo Hostalia](#)

2.4 Varios Ampliación

- Listas `dl`
- Elemento `figure`
- Elemento `audio`
- Elemento `video`
- Elemento `details`

2.4.1 1. Aclarando el uso de las listas `dl`

En HTML 4 las listas `dl` estaba pensadas para su uso con listas de definiciones.

In HTML 4, `<dl>` was considered a "definition list", containing groups of terms and their definitions. The terms and definitions were a many-to-many relationship: one or more terms to one or more definitions.

En HTML 5 su utilización se ha ampliado para desplegar listas de pares llave-valor (key-value).

```
<dl>
  <dt>Authors:</dt>
  <dd>Remy Sharp</dd>
  <dd>Rich Clark</dd>
  <dt>Editor:</dt>
  <dd>Brandon Lennox</dd>
  <dt>Category:</dt>
  <dd>Comment</dd>
</dl>
```

The `dl` element represents an association list consisting of zero or more name-value groups (a description list). A name-value group consists of one or more names (`dt` elements) followed by one or more values (`dd` elements), ignoring any nodes other than `dt` and `dd` elements.

2.4.2 2. El elemento `figure`

El elemento HTML `<figure>` representa contenido independiente, a menudo con un título. Si bien se relaciona con el flujo principal, su posición es independiente de éste. Por lo general, se trata de una imagen, una ilustración, un diagrama, un fragmento de código, o un esquema al que se hace referencia en el texto principal, pero que se puede mover a otra página o a un apéndice sin que afecte al flujo principal.

Ejemplo de `figure` con imagen:

```
<figure>
  
  <figcaption>Fig1. - A view of the pulpit rock.</figcaption>
</figure>
```

Ejemplo de `figure` con una cita:

```
<figure>
  <blockquote>"El ordenador nació para resolver problemas que antes no existían"</blockquote>
  <figcaption>Cita famosa de <cite>Bill Gates</cite></figcaption>
</figure>
```

Tiene más información [en este enlace](#), donde se muestran figuras y distintas posibilidades.

2.4.3 3. El elemento `Audio`

Nota: Antes de HTML 5 el contenido multimedia se reproducía mediante extensiones (plugins: realplayer, flash player...).

```
<audio controls>
  <source src="horse.ogg" type="audio/ogg">
  <source src="horse.mp3" type="audio/mpeg">
  Your browser does not support the audio element.
</audio>
```

Tenga en cuenta:

- Varios elementos `<source>` se enlazan a diferentes ficheros de audio. El navegador usará el primero que encuentre con un formato que pueda reproducir.
- El atributo `controls` añade controles de reproducción como pausa, volumen, etc.
- El texto delimitado por `<audio>` y `</audio>` se muestra si el navegador no soporta el elemento `<audio>`.

Direcciones de dos ficheros de audio para hacer pruebas:

- <https://www.w3schools.com/html/horse.ogg>
- <https://www.w3schools.com/html/horse.mp3>

2.4.4 4. El elemento video

Su uso es similar al del elemento `audio` visto previamente:

```
<video width="320" height="240" controls>
<source src="movie.mp4" type="video/mp4">
<source src="movie.ogg" type="video/ogg">
Your browser does not support the video tag.
</video>
```

Direcciones de dos ficheros de video para pruebas:

- https://www.w3schools.com/html/mov_bbb.mp4
- https://www.w3schools.com/html/mov_bbb.ogg

Tenga en cuenta:

- Varios elementos `<source>` se enlazan a diferentes ficheros de video. El navegador usará el primero que encuentre con un formato que pueda reproducir.
- El atributo `controls` añade controles de video como pausa, volumen, etc.
- El texto delimitado por `<video>` y `</video>` se muestra si el navegador no soporta el elemento `<video>`.
- Es recomendable incluir en este elemento los atributos `width` y `height` para que el navegador conozca el tamaño del vídeo y reserve espacio en pantalla mientras se carga.

2.4.5 5. Elemento details (y summary)

El elemento HTML `<details>` es usado como un *widget de revelación* a través del cual el usuario puede obtener información adicional si está en estado "abierto". Cerrado muestra el contenido del elemento `summary`. Se representa con un triángulo que gira del estado *cerrado* al estado *abierto*.

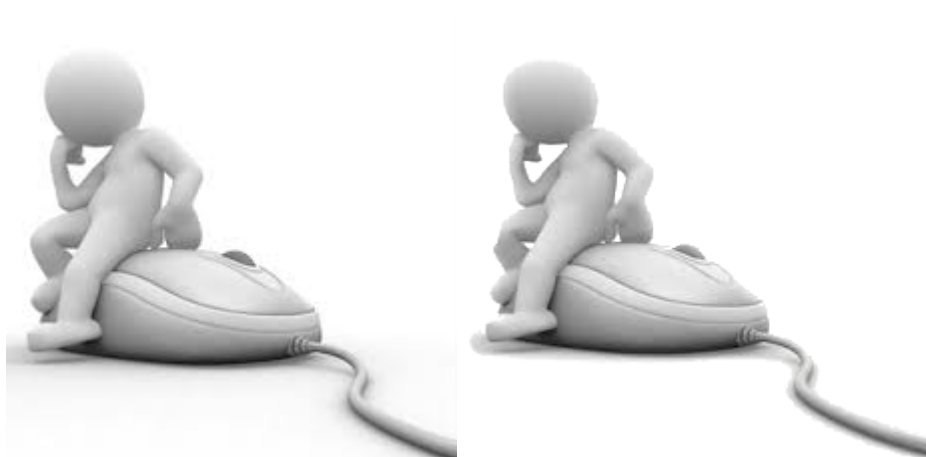
```
<details>
<summary>Copyright 1999-2014.</summary>
<p> - by Refsnes Data. All Rights Reserved.</p>
<p>All content and graphics on this web site are
the property of the company Refsnes Data.</p>
</details>
```

2.4.6 6. Enlaces de consulta

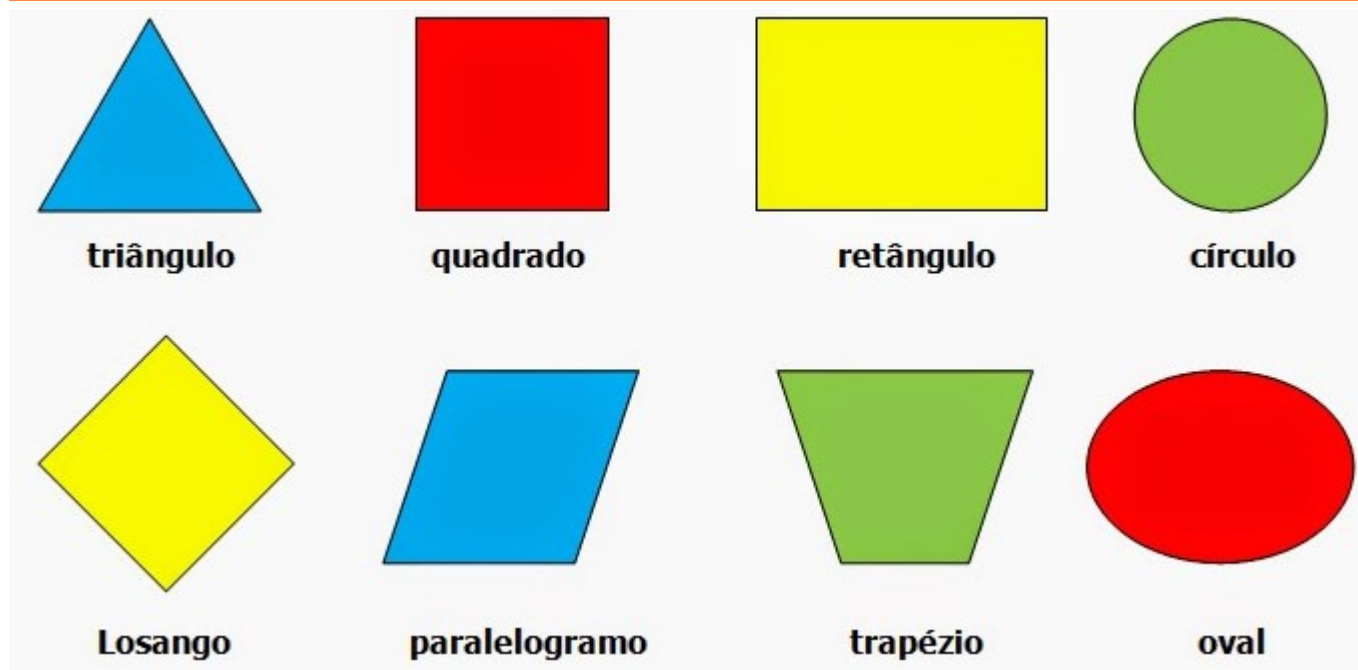
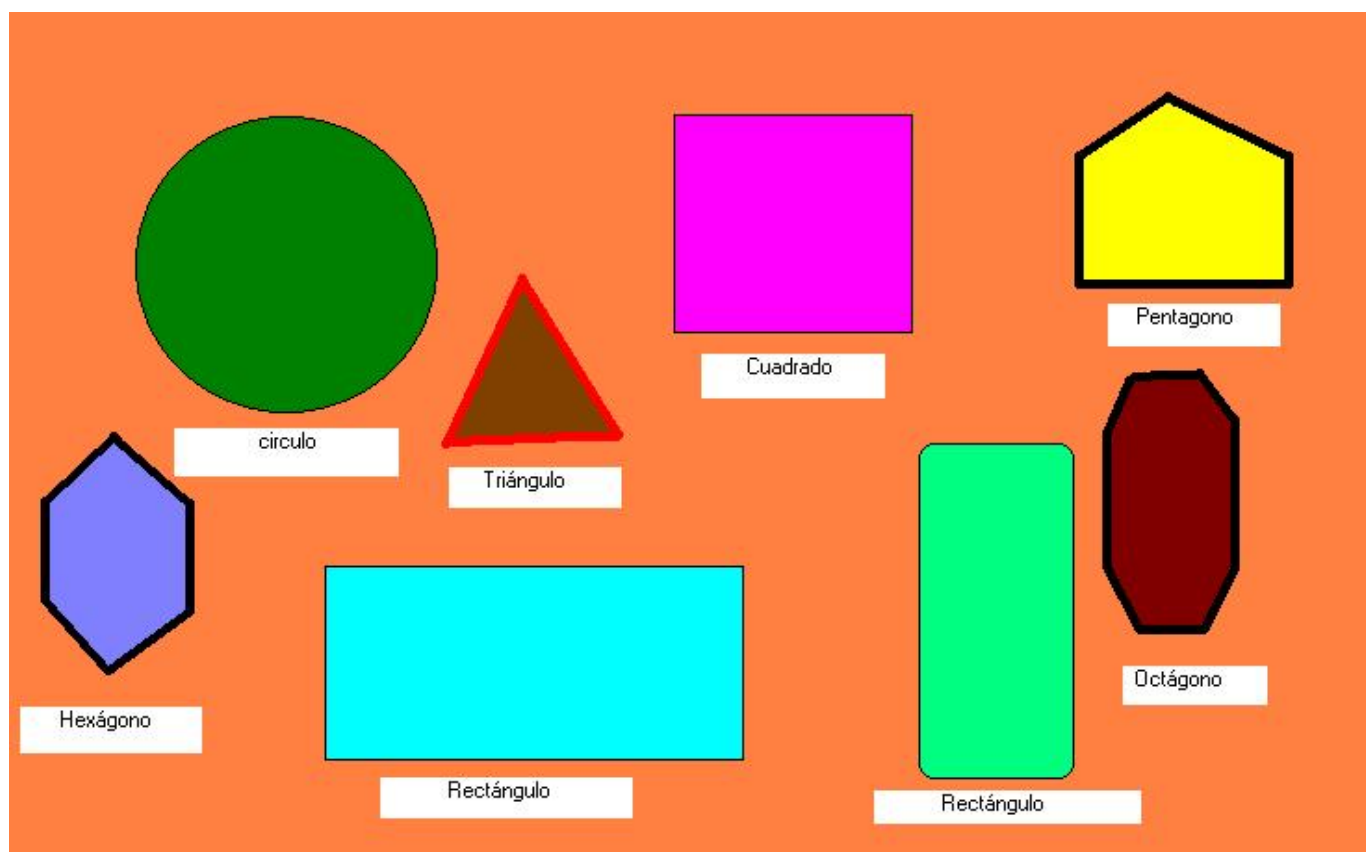
- `dl`: <https://html5doctor.com/the-dl-element/>
- `figure`: <http://html5doctor.com/the-figure-figcaption-elements/>
- `audio`: https://www.w3schools.com/html/html5_audio.asp
- `video`: https://www.w3schools.com/html/html5_video.asp
- `details/summary`: https://www.w3schools.com/tags/tag_details.asp

2.5 imágenes de prueba

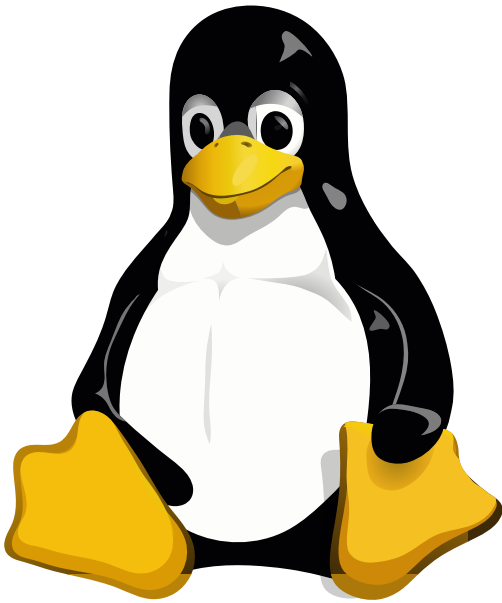
2.5.1 1. Transparencia



2.5.2.2. mapa de imagen



2.5.3 3. vectorial

2.5.4 4. iconos



2.6 Cap. 08

2.6.1 Formularios HTML I

Este documento incluye ejemplos de algunos de los nuevos elementos y atributos para formularios que añade HTML.

- Nuevos tipos de `input`: `email`, `date`, `time`, `tel`, `url`, `number`
- Nuevos atributos o elementos: `placeholder`, `required`, `novalidate`, `max`, `min`, `step`, `multiple`

1. Más tipos de inputs

El *input* de tipo `email` comprueba que el valor que se introduce sigue la sintaxis de un correo electrónico.

```
<label id="correo"> Correo </label> <br>v
<input type="email" name="correo" id="correo" placeholder="Correo corporativo">
```

El *input* de tipo `url` comprueba que el valor que se introduce sigue la sintaxis de una URL.

```
<label id="url"> Página Web </label> <br>
<input type="url" name="url" id="url" placeholder="URL destino">
```

El *input* de tipo `tel` **NO comprueba** que el valor que se introduce sigue la sintaxis de un teléfono, pero podría activar el teclado numérico en un móvil. Si desea controlar el valor introducido puede usar un patrón (`pattern`)

```
<label id="tel"> Teléfono de empresa </label> <br>
<input type="tel" name="tel" id="tel" placeholder="teléfono">
```

El *input* de tipo `number` permite comprobar si es el valor introducido es un número válido. Además podría indicar un valor mínimo (atributo `min`), un máximo (`max`) e incluso el valor de incremento (`step`) que por defecto toma el valor 1.

```
<label id="cantidad"> Cantidad </label> <br>
<input type="number" name="cantidad" id="cantidad" min="2" max="10" step="2" value="6">
```

Si desea números con decimales el atributo `step` debe tomar el valor `any`

```
<label id="precio"> Precio </label> <br>
<input type="number" name="precio" id="precio" step="any">
```

El *input* de tipo `date` comprueba si la fecha es correcta, y el navegador facilita un calendario para seleccionar la fecha.

```
<label id="fechaCompra"> Fecha de compra </label> <br>
<input type="date" name="fechaCompra" id="fechaCompra">
```

El *input* de tipo `time` comprueba si la hora es correcta, y facilita un control para seleccionar.

```
<label id="horaEntrada"> Hora de entrada </label> <br>
<input type="time" name="horaEntrada" id="horaEntrada" value="15:00">
```

2. Más sobre atributos

- El atributo `placeholder` en un *input* es una pequeña ayuda o descripción sobre el valor esperado en dicha entrada del formulario. No confundir con el valor por defecto, que se especifica con `value`.
- El atributo `required` en un *input* hace que el formulario no se envíe si no se introducen datos en ese *input*.
- El atributo `novalidate` en el elemento `form` desactiva la validación local de los input de tipo date, mail, url, etc.
- El atributo `multiple` permite con el tipo file asociar varios ficheros en el input. Y en el tipo email admite varios valores separados por comas.

3. Materiales adicionales

Dentro de la carpeta `cap_08_formularios_ficheros` podrá encontrar:

- El fichero `formulario-htm15.html`: Formulario HTML con ejemplos de algunos nuevos elementos y atributos.
- Fichero `index.html`: formulario básico para prueba junto con el fichero `procesa.php`
- Fichero `procesa.php`: para hacer pruebas con formularios, en un ejemplo de código PHP que muestra los parámetros recibidos.

Aviso: Para poder probar el código PHP debe seguir las instrucciones que se adjuntan en la plataforma.

4. Enlaces de consulta

- [Los nuevos atributos HTML5 para inputs y formularios](#), por Aner Barrena
- [Elementos de formulario adicionales del HTML5](#), por el W3C.
- [Diferencias entre métodos get y post](#).
- Tiene un enlace en la plataforma al fichero *PHP-ciclo-server.pdf* con una breve explicación sobre el funcionamiento de un servidor PHP.

2.6.2 Formularios HTML II

Tiene el código de estos elementos en los ficheros de ejemplos.

1. Range

El Input de tipo `range` es similar a `number` pero proporcionando un control gráfico:

```
<label id="volumen"> volumen </label> <br>
<input type="range" name="volumen" id="volumen" min="0" max="10" step="1" value="5">
```

2. Color

El input de tipo `color` muestra un selector para elegir color. Se envía el código hexadecimal que le corresponde:

```
<label id="colorFondo"> Color de fondo </label> <br>
<input type="color" name="colorFondo" id="colorFondo" value="#ffff00">
```

3. Patrones

El atributo `pattern` se le puede añadir a un input de tipo texto y restringe los posibles valores de la entrada.

En este ejemplo se fuerza a un formato para un DNI: 9 dígitos seguidos de una letra mayúscula entre A y Z. Estudiará patrones con más detenimiento en otros módulos.

```
<label id="dni"> NIF </label> <br>
<input type="text" name="dni" pattern="[0-9]{8}[A-Z]" id="dni">
```

Enlace sobre [expresiones regulares](#)

4. datalist

El elemento `datalist` y su atributo asociado `list` sirven para presentar al usuario una lista de sugerencias de valores (no limita, se puede enviar otro valor no incluido en la lista).

El `input` y la `datalist` se relaciona con el atributo `list` en el `input` y el `id` en el `datalist`.

```
<label for="from"> Origen: </label> <br>
<input name="from" type="text" list="city" id="from">
<datalist id="city">
  <option value="Adelaide">Adelaide
  <option>Beijing
  <option>Bangkok
  <option>Hanoi
  <option>Hong Kong
</datalist>
```

5. search

El input de tipo `search` es similar al de texto. Algunos agentes de usuario cambian la forma de mostrarlo, o muestran entradas anteriores.

```
<label id="buscar"> NIF </label> <br>
<input type="search" name="buscar" id="buscar" >
```

6. Tipo file y atributo accept

El atributo `accept` toma como valor una lista separada por comas de uno o más tipos de archivos, o especificadores de tipo de archivo únicos, que describen qué tipos de archivo permitir.

Ejemplo de uso:

```
<input type="file" name="resumen"
  accept=".doc,.docx,application/msword" />
```



```
<input type="file" name="soundFile" accept="audio/*" />
<input type="file" name="videoFile" accept="video/*" />
<input type="file" name="imageFile" accept="image/*" />
```

7. number y decimales

En el tipo `number` además del valor `any` puede especificar el salto en los decimales, por ejemplo `step="0.01"`. Esto permite hasta dos dígitos en la parte decimal con incrementos de una centésima.

```
<input type="number" name="precio" step="0.01" min="0" />
```

8. Varios

- El atributo `autofocus` es un atributo del tipo lógico. Su existencia en un input indica que dicha entrada deberá obtener el foco cuando la página se muestra.
- No todos los navegadores soportan siempre todos los inputs, . Por ejemplo, los inputs `date`, `time` o `color` no se interpretan correctamente en Internet Explorer. En estos caso de inputs no conocido se interpretan como de tipo `text`, que es el valor por defecto de un input.
- Existen otros inputs relacionados con `date` y `time`: `datetime`, `week`, `month`, etc.

2.6.3 Server PHP y linux

1. Instalar server PHP

Puede utilizar el servidor de pruebas que incorpora el paquete PHP (comando `php -S`).

```
$ php -S localhost:8000
[Wed Jun 25 20:23:41 2025] PHP 8.2.28 Development Server (http://localhost:8000) started
...
```

Si no estuviese instalado basta con instalar el paquete `php-cli`:

```
$ sudo apt update
$ sudo apt install php-cli
...
$ php --version
PHP 8.2.7 (cli) (built: Jun 9 2023 19:37:27)
Copyright (c) The PHP Group
...
$
```

2. Servidor PHP en línea de comandos

Para probarlo crear en el directorio del proyecto el fichero `index.php`:

index.php

```
<?php
    phpinfo();
?>
```

Lanzar el servidor de pruebas PHP con el comando `php -S <IP>:<puerto>`

```
$ php -S localhost:5000
[Thu Oct 20 12:08:04 2022] PHP 7.4.30 Development Server (http://localhost:5000) started
```

Abrir el navegador con la URL `http://localhost:5000` (por ejemplo desde otra terminal):

```
firefox http://localhost:5000 &
```

En el navegador se muestra una tabla con información sobre PHP. Y en la consola del servidor le aparece el log con las peticiones que se realizan:

```
$ php -S localhost:5000
[Thu Oct 20 12:08:04 2022] PHP 7.4.30 Development Server (http://localhost:5000) started
[Thu Oct 20 12:17:23 2022] [::1]:47404 Accepted
[Thu Oct 20 12:17:23 2022] [::1]:47404 [200]: (null) /
[Thu Oct 20 12:17:23 2022] [::1]:47404 Closing
```

Tenga en cuenta que **por defecto la raíz del sitio web será el directorio desde dónde se ha iniciado el servidor PHP**.

3. Plugin en VSCode

Una vez instalado `php-cli` puede instalar el plugin **PHP Server** de **brapifra** que se basa en el servidor PHP instalado en el sistema. Tenga en cuenta que no es un "live server"

Para lanzar el servidor debe usar la opción **"PHP Server: Serve Project"**

2.7 Errores frecuentes Bloque HTML

2.7.1 Estructura

- Código HTML fuera del `body` (o en `head`)
- No especificar `title` significativo del documento.
- Falta el idioma del documento en atributo `lang`.
- No identificar los bloques `aside`, `footer`, `article`, `nav`, etc. (no tienen por qué estar todos).

2.7.2 Texto

- Texto flotante: texto no delimitado por ninguna etiqueta.
- Uso del elemento `<br>` fuera de formularios.
- Confundir `code` y `pre`.
- No usar correctamente `code` y `pre`.
- No usar entidades `<`, `>`, etc. al mostrar código con etiquetas HTML.
- Olvidar que `abbr` y `dfn` llevan atributo `title`.

2.7.3 Cabeceras

- Falta `<h1>`
- Hay más de un `<h1>`
- Cabecera de un bloque fuera del mismo: si hay `section`, `article`, `footer` o similar la cabecera que da título a ese bloque va dentro, no fuera.
- No puede haber saltos "hacia abajo" en los niveles de las cabeceras: por ejemplo, después de un `h2` no puede haber un `h4` => usar complemento del navegador `headingsMaps`.

2.7.4 Listas

- No anidar correctamente: la lista interior debe estar incluida totalmente en el `li` superior.

```
<ul> <!-- CORRECTO -->
  <li> Apartado 1    <!-- Apertura li apartado 1 -->
    <ul> <!-- inicio lista anidada -->
      <li> Subapartado 1.1</li>
      <li> Subapartado 1.2</li>
    </ul> <!-- cierre lista anidada -->
  </li> <!-- Cierre li apartado 1 -->
  <li> Apartado 2 </li>
</ul>
```

- No acordarse de que existen las listas de definiciones: `dl>dt+dd`
- Si tengo una lista de enlaces, los enlaces van dentro del `li`, no al revés.

```
<!-- incorrecto -->
<a href="destino,html"><li>Enlace a destino </li></a>
<!-- correcto -->
<li><a href="destino,html">Enlace a destino </a></li>
```

2.7.5 Imágenes

- Una imagen tiene **4 atributos**: `src`, `alt`, `height` y `width` (recordar ayuda editor con tamaños)
- El texto `alt` debe ser significativo
- Existe un elemento `figure` que a veces hay que usar, pero que no es exclusivo de imágenes.

2.7.6 Formularios

- Olvidar elemento `form` que envuelve a todos los inputs o colocar los inputs fuera del formulario
- Olvidar atributo `action` o usar valor incorrecto.
- No usar correctamente el `method` `get` o `post` según situación.
- Recordar que el uso de un input de tipo `file` obliga a usar el método `post` y atributo `enctype="multipart/form-data"`
- No usar bien el label: input sin su label bien asociada. ¡¡**Probar en el navegador!!**
- Usar input sin atributo `name` => el input ¡¡no se envía!!.
- No usar los tipos adecuados en los input (en función del valor a enviar: email, url, date, number, etc).
- Olvidar que inputs como `select`, `radio` o `checkbox` deben especificar los valores de cada opción.
- No usar atributos como `required`, `placeholder`, `pattern`, etc.
- El input de tipo `radio`:
 - error: poner names distintos en los cada elemento del grupo `radio`
 - error: no fijar `value` en cada elemento.
 - error: no asociar bien el `label` de cada input
- Error: **No probar** el formulario: evitaría la mayoría de los errores anteriores.

2.7.7 Tablas

- El `caption` se coloca justo a continuación de table.

```
<table>
  <caption> Resultado trimestrales </caption>
  <tr>
    ....
```

- Las cabeceras de fila o columna se etiquetan como `th`, y se especifica su `scope` con los valores `row` o `col`.
- Si la tabla tiene filas finales que actúan como pie de tabla o la cabecera de la fila son varias filas se pueden usar las etiquetas `thead`, `tfoot`, `tbody`.

2.7.8 Enlaces y URLs

- Error: En el examen se pide que en la barra de navegación se usen enlaces relativos y no se pueden usar los enlaces relativos a la raíz del sitio (los que comienzan con `/`).
- Si las direcciones son externas deben comenzar con `http://` o `https://`
- Las url pueden tener una parte final, la query, donde se especifican los campos que se le pasan a la página destino.
- Imágenes que son enlaces: envolver adecuadamente la imagen con la etiqueta `a`.

```
<a href="destino.html">  </a>
....
```

- Enlaces a partes concretas de una página: identificar destino con `id="valor"` y enlazar con `#valor`.

```
...
<li><a href="#apartado01">Apartado 01 </a></li>
...

<h2 id="apartado01">Apartado 01: enlaces relativos <h2>
...
```

2.8 Varios

2.8.1 Curiosidades

- [La historia del estándar HTML](#)

2.8.2 Emmet

- [Emmet Cheat Sheet](#)

2.8.3 Banco imágenes gratuitas

- Banco de iconos: <https://iconarchive.com/>
- Imágenes aleatorias: <https://picsum.photos/>
- Los 50 mejores bancos de imágenes, fotos e iconos GRATIS: <https://aulacm.com/bancos-de-imagenes-gratis/>
- Stockvault: <http://www.stockvault.net/>

2.8.4 Mapas de imagen online

- <https://imagemap.org/>
- <https://www.image-map.net/>

Los editores gráficos suelen incluir herramientas que simplifican la creación de un mapa de imagen. Por ejemplo, existe un filtro en gimp (<https://docs.gimp.org/es/plugin-imagemap.html>) o photoshop (https://www.aulaclic.es/photoshopcs2/t_14_3.htm) también lo permiten.

2.8.5 El mundo del favicon

- Adding favicons in a multi-browser multi-platform world: <https://mobiforge.com/design-development/adding-favicons-in-a-multi-browser-multi-platform-world>
- A comprehensive cheat sheet to favicon sizes/types: <https://github.com/audreyfeldroy/favicon-cheat-sheet>
- Mclibre favicon: <https://www.mclibre.org/consultar/htmlcss/html/html-favicon.html>
- Favicon generator: <https://realfavicongenerator.net/>
- Más sobre favicon: edición <https://raiolanetworks.es/blog/favicon>

3. 02. CSS

3.1 Introducción

Tiene un [esquema en este enlace](#).

Este bloque está compuesto por:

1. Los apuntes de Uniwebsidad:

- Aquí una [copia en PDF](#)
- o en este enlace <https://web.archive.org/web/20241207111819/https://uniwebsidad.com/libros/css>

2. Los complementos de esta llamada ZonaWeb

3.2 Ampliación Selectores Uniwebsidad

Tiene una tabla resumen con ejemplos en W3schools: http://www.w3schools.com/cssref/css_selectors.asp

3.2.1 1. Selectores de atributos

Además de los ya vistos en los apuntes de uniwebsidad tenemos:

- `[attribute^=value]` Comienza por.. Por ejemplo, enlaces seguros: `a[href^=https://]`
- `[attribute$=value]` Termina por.. Por ejemplo, enlaces pdf: `a[href$=".pdf"]`
- `[attribute*=value]` Contiene a.. `img[alt*="logo"]`

3.2.2 2. Selector sibling (~)

```
element1~element2 { /* General sibling combinator*/
```

Este selector `element1~element2` se aplica cuando el elemento2 está precedido por el elemento1. Ambos deben tener el mismo padre, pero el elemento2 no tiene porqué estar justo a continuación del elemento1.

La diferencia con el selector adyacente es esa, puede haber elementos intermedios entre elemento1 y elemento2.

3.2.3 3. Selectores estructurales `:nth`

3.1. `:nth-child(an+b)`

Selecciona los elementos (sin importar el tipo) que sean el hijo "n" de su padre. El valor de n puede ser un número, una palabra clave (odd o even) o una fórmula.

En el siguiente ejemplo se hace referencia al tercer elemento de listas no numeradas:

```
ul li:nth-child(3) {
  color: blue;
}
```

Teniendo en cuenta que el índice del primer elemento es uno, las palabras reservadas odd (impar) y even (par) se pueden usar en vez de un valor numérico. El siguiente ejemplo selecciona las filas pares de una tabla:

```
tr:nth-child (even) { /* Selecciona filas pares*/
  background-color: silver;
}
```

Con este selector también se puede usar la *fórmula* (an + b): a representa el tamaño del ciclo y b es el valor de desplazamiento inicial:

```
/* Selecciona cada tres filas, el segundo_ */
tr:nth-child(3n+2) {
  background-color: silver;
}
```

3.2. `:nth-last-child(an+b)`

Similar al anterior, pero se empieza a contar por el último elemento del contenedor

3.3. Atajos `:nth-child`

3.3.1. `:first-child`

Equivalente a `:nth-child(1)`

```
p:first-child {
  background-color: yellow;
}
```

3.3.2. `:last-child`

Equivalente a `:nth-last-child(1)`

```
p:last-child {
  background: #ff0000;
}
```

3.4. `:nth-of-type(an+b)`

Numera los elementos hijos de un determinado tipo, y selecciona aquellos que cumplan estar en la posición "n". El valor de n puede ser un número, una palabra clave (odd o even) o una fórmula, de la misma forma que el selector `nth-child()`.

Su uso es recomendado si en el padre, el contenedor, hay elementos de distintos tipos. Por ejemplo, si queremos de color plata los párrafos pares de un artículo, y como es posible que dentro del artículo haya imágenes, citas, cabeceras, usamos `nth-of-type()` en vez de `nth-child()`

```
article p:nth-of-type(even) {
  color: silver;
}
```

Tiene ejemplos de la diferencia entre `nth-child` y `nth-of-type` en los ejemplos de clase, y adicionalmente en el fichero `css_selectores_nth_ejemplo.html` de la carpeta de ejemplos de clase.

3.5. `:nth-last-of-type(an+b)`

Similar a `:nth-of-type` pero comienza a contar por el último

3.6. Atajos `nth-of-type`3.6.1. `:first-of-type`

Es equivalente a `:nth-of-type(1)`, selecciona el primer elemento de ese tipo para cada padre.

```
p:first-of-type {
  background: red;
}
```

3.6.2. `:last-of-type`

Es equivalente a `:nth-last-of-type(1)`.

```
p:last-of-type {
  background: #ff0000;
}
```

3.7. Otras posibilidades

3.7.1. A PARTIR DE UN ELEMENTO

```
ul li:nth-child(n+3) {
  color: blue;
}
```

3.7.2. HASTA UN ELEMENTO

```
ul li:nth-child(-n+3) {
  color: blue;
}
```

3.2.4 4. Especificidad CSS: resolviendo colisiones

En los apuntes de uniwebsidad no se detalla **cómo** se determina que un selector es más específico que otro, y a veces nos podemos llevar sorpresas.

Simplificando, será más específica:

- La regla con más selectores de ID.
- En caso de empate la que tenga más selectores de clase (o de atributo o pseudo-clases).
- Y en caso de empate la que tenga más selectores de tipo (y pseudo-elementos).
- Y si sigue el empate, la última.

Por ejemplo

```
.resumen { ... }
```

es más específico que

```
section > article h2 + p { ... }
```

Nota: Recuerde que Visual Code le muestra la especificidad de un selector al situar el ratón sobre el mismo.

Aquí tiene algunos enlaces adicionales:

- Calculadora: <https://specificity.keegan.st/>
- Una breve explicación en castellano: <https://blog.outbook.es/desarrollo-web/css-especificidad-de-los-selectores>
- Lo que dice la norma: <https://www.w3.org/TR/selectors-3/#specificity>

3.3 Colores

3.3.1 1. Opacidad

La propiedad CSS `opacity` define la transparencia de un elemento, esto es, en qué grado se superpone el fondo al elemento.

```
p {
  color: blue;
  opacity: 0.4; /* valor entre 0 y 1 */
}
img.difumina {
  opacity: 0.2;
}
```

Es un número cuyo valor se encuentra entre 0.0 y 1.0, ambos incluidos. Este valor representa la opacidad.

- 0: el elemento es transparente (invisible).
- Cualquier valor entre 0 y 1: El elemento es translúcido.
- 1: el elemento es opaco (sólido).

3.3.2 2. Colores hsl

HSL es el acrónimo de hue(matiz), saturation (saturación), lightness (luminosidad).

- El valor del matiz se refiere a la frecuencia dominante del color dentro del espectro visible. Es un valor entre 0 y 360 (una rueda de color): rojo vale 0, verde 120, azul 240.
- La saturación se refiere a la intensidad del color, a la «pureza» de éste. 0% means a shade of gray and 100% is the full color.
- La luminosidad es la cantidad de luz de un color. S y L son porcentajes entre 0 y 100%: 0% is black, 100% is white.

```
section {
  background-color: hsl(180, 50%, 50%);
  color: hsl(0, 0%, 100%);
}
```

Usar HSL hace más sencillo crear armonías de colores variando alguno de los tres valores: paletas monocromáticas, paletas de colores análogos, complementarios, triadas, etc. Puedes hacerte una idea en estas webs:

- <http://www.colorhexa.com/>
- <http://htmlcolorcodes.com/color-picker/>
- <http://paletton.com/>

3.3.3 3. Añadir opacidad al color: hsla y rgba

La opacidad se puede especificar al mismo tiempo que el color. Para ello se usan las propiedades rgba, y hsla, que reciben un 4º valor, la opacidad. Ejemplos:

```
p.difumina {
  color: rgba(0, 0, 255, 0.5);
}

h2 {
  color: hsla(0, 100%, 50%, 0.5);
}
```

Nota: incluso es posible encontrarlo en representación hexadecimal, añadiendo un cuarto octeto para la opacidad:

```
p.difumina {
  color: #0000FF80;
}
```

3.4 Box model

3.4.1 1. Border-radius

Redondea las esquinas del `border` de un elemento. Se pueden especificar entre 1 y 4 valores y se interpretan de la misma forma que en el `padding` o `margin`:

```
section {
  border: 1px solid blue;
  border-radius: 15px; /* se pueden especificar 1, 2, 3 o 4 valores */
  padding: 15px;
}
```

<https://css-tricks.com/almanac/properties/b/border-radius/>

3.4.2 2. Box-shadow

Se especifica el desplazamiento horizontal, el vertical, el “blur” o desenfoque y el color de la sombra:

```
section {
  border: 1px solid blue;
  box-shadow: 40px 30px 10px #444;
  margin: 40px;
}
```

Incluso se pueden especificar varias sombras (separadas por comas):

```
#example1 {
  border: 1px solid;
  padding: 10px;
  box-shadow: 5px 5px blue, 10px 10px green;
}
```

2.1. text-shadow

No tiene nada que ver con el modelo de cajas, pero añadimos esta propiedad en este documento por ser su uso muy parecido a `box-shadow`:

```
h1 {
  text-shadow: 2px 2px 1px #666;
}
```

3.4.3 3. box-sizing

Permite que al tener en cuenta la propiedad `width` (y `height`) se incluya el valor del `padding` y el `border` juto con el del contenido (no el margen). Esto permite aplicar anchos por ejemplo del 50% sin tener que considerar la anchura del `padding` o del borde. Es posible aplicarla todos los elementos con el operador universal:

```
* {
  box-sizing: border-box;
}
```

o sólo aplicarla a los elementos necesarios en cada caso, por ejemplo:

```
header, nav, aside, footer, section {
  box-sizing: border-box;
}
```

+info: https://www.w3schools.com/cssref/css3_pr_box-sizing.asp

3.4.4 4. Multiple Backgrounds

Se pueden especificar varias imágenes de fondo en un mismo elemento. Para ello basta con usar la propiedad `background` con los valores de cada una de ellas separados por comas. La imagen que se especifica primero es la que aparece más cercana al usuario:

```
#example1 {  
  background: url(img_flwr.gif) right bottom no-repeat,  
             url(paper.gif) left top repeat;  
}
```

O especificando cada una de las propiedades que agrupa `background`:

```
body {  
  background-image: url("img_tree.gif"), url("paper.gif");  
  background-repeat: no-repeat, repeat;  
  background-color: #cccccc;  
}
```

En este enlace tiene un ejemplo de uso:

- https://www.w3schools.com/css/tryit.asp?filename=trycss3_background_multiple

3.5 CSS y texto

3.5.1 1. text-decoration

En la web de Uniwebsidad no queda claro el uso de `text-decoration`. Realmente es un *shorthand* de varias propiedades. Además permite modificar el color, estilo y el grosor.

```
section strong {
  /* ejemplo inicial */
  /* text-decoration: underline; */

  /* ejemplo "ampliado" */
  text-decoration: red solid underline 2px;
  /* que equivale a
    text-decoration-color:red ;
    text-decoration-style: double;
    text-decoration-line: underline;
    text-decoration-thickness: 2px */
}
```

3.5.2 2. Web-fonts

En la carpeta de fuentes existe una carpeta de nombre `t06_css_webfonts` donde se explica y aplica el uso de web fonts y algunas propiedades adicionales.

Tenemos dos formas de incorporar fuentes adicionales en nuestro fichero CSS:

- Fuentes en un fichero local
- Fuentes web u online

2.1. Fuente en fichero local

Si disponemos de un fichero con la fuente a utilizar se necesita:

- Una directiva `@font-face` (en el fichero CSS) que le da un nombre a la nueva fuente y asocia dicho nombre al fichero.
- Luego usaremos esa fuente con el nombre indicado en el paso previo

```
@font-face {
  font-family: fNovedades; /* asociamos el nombre fNovedades al fichero siguiente */
  src: url("mona_shark/Mona Shark.otf");
}

.novedades {
  font-family: fNovedades, serif; /* mismo nombre usado en @font-face font-family */
}
```

2.2. Fuentes web

Para incluir fuentes de Google (o de otros proveedores) accedemos a la url <https://fonts.google.com/> que nos ofrece esas fuentes.

Si pulsamos sobre una fuente nos da la opción de descargarla (para usarla como en el caso anterior). Nosotros en este caso vamos a seleccionarla (*Select this Style*) y nos aparece a la derecha las instrucciones para utilizarla en nuestra página sin tener que descargarla. Por ejemplo, seleccionada la fuente `Ole` debemos dar dos pasos:

1. Incluir los enlaces que nos propone en el head del fichero HTML. Serán similares a estos:

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<!-- Aquí van las fuentes -->
<link href="https://fonts.googleapis.com/css2?family=Press+Start+2P&family=Roboto&display=swap" rel="stylesheet">
...
<!-- hoja de estilos del usuario -->
<link href="misestilos.css" rel="stylesheet">
```

2. Y usar la propiedad `font-family` en el fichero `misestilos.css` como se observa en este ejemplo

```
p {
  font-family: 'Ole', cursive;
}

h2 {
  font-family: 'Roboto', sans-serif;
}
```

2.3. Enlaces relacionados

- Google Fonts: <https://fonts.google.com/>
- Fuentes:
- CDN Fonts: <https://www.cdnfonts.com/>
- The League of Moveable Type: <https://www.theleagueofmoveabletype.com/>
- Font Space: <https://www.fontspace.com/>

3.5.3 3. pseudoelementos

En el tema 6 se introducen los pseudo elementos `::first-line` y `::first-letter`. Completamos ese apartado con otros dos, `::after` y `::before`, con la propiedad `content` y el método `attr()`.

Por lo tanto en este apartado estudiaremos:

- `::after`
- `::before`
- `content`
- `attr()`

3.1. `::after`, `::before` (y `content`)

Para insertar contenido generado desde la hoja de estilo delante (`::before`) o detrás (`::after`) de un elemento debemos usar la propiedad `content`.

En este ejemplo se añade delante de cada cabecera de nivel 1 el texto "Capítulo -"

```
h1::before {
  content: "Capítulo - ";
}
```

3.2. `content` y `attr()`

Además podemos usar `content` junto con la función `attr()` para mostrar el valor de un atributo del elemento (que de otra forma no se muestra en pantalla). Por ejemplo se puede usar para mostrar el significado de las abreviaturas, elemento `abbr` con atributo `title`:

```
abbr::after {
  content: " (" attr(title) ") ";
}
```

O mostrar la URL de los enlaces tras el texto del enlace.

```
a[href]:after {
  content: " (" attr(href) ") ";
}
```

Observe que en este último ejemplo se ha usado `:after` (obsoleto) en vez de `::after`. La notación `::` para los pseudo elementos se añadió más tarde y los navegadores soportan las dos para mantener compatibilidad hacia atrás.

3.5.4 4. Hojas de estilo para impresión

Los estilos para impresión se pueden añadir de dos formas:

- Añadiendo un fichero CSS con los estilos que se deben aplicar al imprimir, y al enlazarlo indicar que el destino (`media`) es la impresora. Los valores más usados para `media` son `all` (valor por defecto), `screen` (pantalla) y `print` (impresora).

```
<link rel="stylesheet" href="general.css">
<link rel="stylesheet" href="impresora.css" media="print">
```

- Añadiendo los estilos de impresión en el fichero de estilos general usando la directiva `@media print{}`. En este caso quedaría:

```
@media print {
  a[href]:after {
    content: " (" attr(href) ") ";
  }
}
```

Para ver el resultado y las reglas CSS que se aplican en el inspector de elementos (F12 o CTRL+MAY+I) debe acceder al mismo y pulsar sobre el icono "alternar simulación de medios de impresión para la página" disponible en la zona de estilos.

Otra opción para ver cómo se aplican los estilos de impresión es usar la opción de imprimir, aunque debe tener precaución porque los navegadores permiten configurar algunas opciones de estilos, como por ejemplo los colores o imágenes de fondo, que por defecto aparecen desactivadas para que no se muestren.

Puede encontrar más detalles en este enlace: <https://css-tricks.com/can-you-view-print-stylesheets-applied-directly-in-the-browser/>

3.5.5 5. Contadores CSS

Tiene varios ejemplos en el directorio `t06_css_contadores`

5.1. Uso básico

El uso de contadores se basa en el pseudo elemento `::before` (o `::after`), la propiedad `content` para insertar el contador y el uso de estas nuevas propiedades:

- `counter-reset` - crea o reinicia un contador
- `counter-increment` - incrementa el valor de un contador
- `counter()` - muestra el valor de un contador.

Por ejemplo, si desea numerar los párrafos de una sección (ver ejemplo `css_contadores_01.html`)

```
section { /* modificar section/article para ver la diferencia */
  counter-reset: np;
}
section p {
  counter-increment: np;
}
section p::before {
  content: "(" counter(np) ") ";
}
```

5.2. Contadores en cabeceras

Si va a numerar las cabeceras de nivel 2, debe iniciar el contador en una regla CSS asociada al elemento padre de esas cabeceras, e incrementarlo y mostrarlo en una propiedad del elemento a numerar usando la pseudoclase `::before`

Por ejemplo, si quiere numerar las cabeceras de nivel 2 de una sección de la clase `unnivel`

```
section {
  counter-reset: nh2;
}
section h2 {
  counter-increment: nh2;
}
section h2::before {
  content: "Apartado " counter(nh2) ": ";
}
```

5.3. Contadores en listas anidadas

Si tenemos listas anidadas

```
<ol>
  <li>Lorem.</li>
  <li>Quaerat
    <ol>
      <li>Lorem, ipsum.
        <ol>
          <li>Lorem, ipsum dolor.</li>
          <li>Assumenda, minus ipsam!</li>
        </ol>
      </li>
      <li>Ipsam, quasi!</li>
      <li>Fuga, commodi.</li>
    </ol>
  </li>
  <li>Sit.</li>
</ol>
```

podemos usar la función `counters` (ojo a la `s` final). Esta función recibe dos argumentos: el contador y el separador a usar si hay varios contadores con el mismo identificador:

```
ol {
  counter-reset: nli;
  list-style-type: none;
}
li::before {
  counter-increment: nli;
  content: counters(nli, ".") " ";
}
```

5.4. Ejercicios

1. Revisar ejemplos de código proporcionados en la plataforma
2. Hacer ejercicio propuesto en ejemplos de código proporcionados en la plataforma

5.5. Enlaces de ampliación

Puede obtener más información en:

- W3Schools: Contadores CSS https://www.w3schools.com/css/css_counters.asp
- MDN: Using CSS counters https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Counter_Styles/Using_CSS_counters
- Ejemplos <https://www.smashingmagazine.com/2013/04/css-generated-content-counters/>

3.5.6 6. Fuera de temario: Familias de fuentes en Linux

6.1. Enlaces

- <https://geekland.eu/instalar-fuentes-en-debian-linux/>

- <https://h4ckseed.wordpress.com/2022/01/31/how-to-gestion-de-fuentes-en-gnu-linux/>

6.2. Instalar las fuentes de Microsoft o variantes

En el caso que queramos o tengamos necesidad de instalar algunas de las fuentes de *Microsoft*, tan solo tenemos que abrir una terminal y ejecutar el siguiente comando:

```
sudo apt-get install ttf-mscorefonts-installer
```

Una vez instalado el paquete `ttf-mscorefonts-installer` pasaremos a disponer de las siguientes fuentes:

Andale Mono, Arial Black, Arial, Comic Sans MS, Courier New, Georgia, Impact, Times New Roman, Trebuchet, Verdana, Webdings.

La totalidad de fuentes mencionadas son fuentes privativas que se hallan en el repositorio Contrib de Debian.

Existen variantes libres de las tipografías Times, Arial y Courier, etc. Se pueden instalar el paquete `fonts-liberation` ejecutando el siguiente comando en la terminal:

```
sudo apt-get install fonts-liberation
```

6.3. Gestionar las fuentes

Puede usar `font-manager` y sólo tienen que ejecutar el siguiente comando en la terminal:

```
sudo apt-get install font-manager
```

En el caso que precisen sacar un listado de la totalidad de fuentes instaladas, pueden ejecutar el siguiente comando en la terminal:

```
fc-list
```

TODO: Algunas fuentes tienen un mapeado, por ejemplo Times a Nimbus Roman o Arial a Liberation Sans.

3.6 Layout HTML con CSS `grid`

3.6.1 1. Código ejemplos *layout*

Los ejemplos están disponibles en el directorio `t12_01_layout`. Para probarlos basta modificar el fichero CSS enlazado en el fichero `layout.html`:

- Usando `grid`: `grid.css`
- Usando `grid` con nombres de áreas: `grid_areas.css`

En la carpeta `otros` tiene otras posibilidades que no son parte de los contenidos evaluables:

- Usando `flex`: `flex.css` (no tanto para layout sino para el flujo en una única línea)
- Usando `float`: `float.css` => obsoleto pero se encuentra todavía en uso.
- Usando `display: table-cell`: fichero `table-cell.css` => obsoleto
- Usando `display: inline-blocks` fichero `inline-blocks.css` => obsoleto

Por lo tanto en este tema **nos vamos a centrar en el uso de `grid` y `grid areas`**.

3.6.2 2. *Layout* con `grid`

Al aplicarle la propiedad `display:grid` a un elemento HTML lo convertimos en un `grid` o rejilla. Por defecto esa rejilla tiene una única columna, y **cada elemento hijo de la rejilla** se dispone en una celda (en el orden que aparecen en el fichero HTML).

Con la propiedad `grid-template-columns`: se define el número y tamaño de las columnas de la rejilla.

Modificación 2025: se añaden carpetas con ejemplos muy básicos del uso de `grid`.

- Carpeta 01: ejemplo del uso de `grid`, `grid-template-columns` y `gap`
- Carpeta 02: ejemplo del uso de `grid-template-areas` y `grid-area`
- Carpeta 03: ejemplo en el que se muestra que en algunas situaciones es más sencillo usar `grid-columns` que `grid-area`

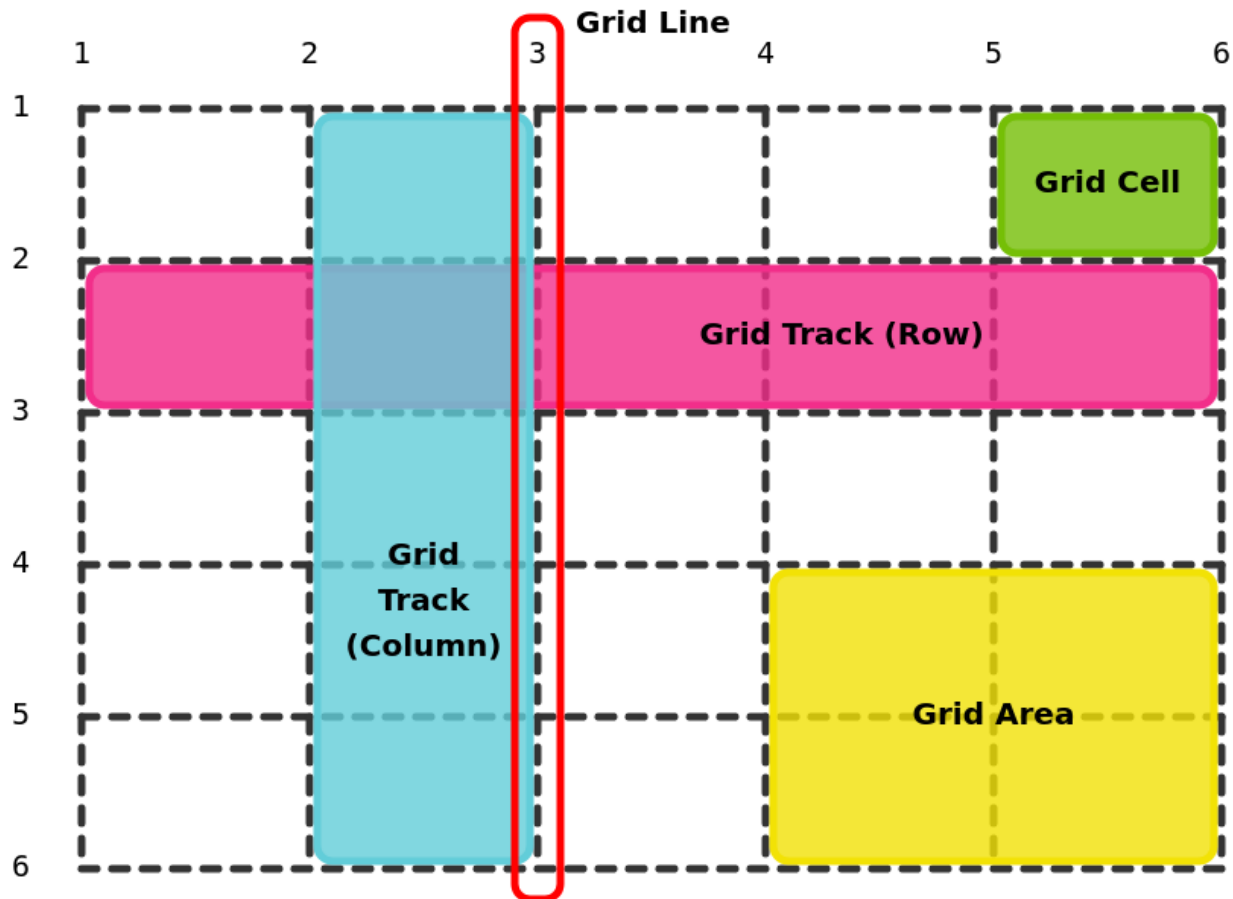
Se recomienda analizarlos antes de seguir.

3.6.3 3. Layout con posicionamiento filas y columnas

Las líneas que delimitan la rejilla se numeran a partir de 1. Si hemos definido 3 columnas, las líneas irían de la 1 a la 4. Esos números de líneas se usan en las propiedades `grid-column` y `grid-row` para indicar:

- donde comienza / donde termina una celda (p.e. `grid-column: 1 / 4`; indica que comienza en la línea 1 y termina en la 4)
- o dónde comienza y a cuantas cuadrículas se expande una celda (p.e. `grid-column: 1 / span 3`; significa que comienza en la línea 1 y ocupa el espacio de tres celdas)

En la siguiente figura puede observar una rejilla y marcadas las líneas, celdas, filas y columnas.



En el código de ejemplo tiene una explicación de:

- dónde definir el grid (`display: grid`)
- cómo definir el número de columnas (`grid-template-columns`) con porcentajes o con la unidad `fr`.
- Y como situar cada bloque en el grid (propiedades `grid-column` y `grid-row`), ya sea dejando su posición por defecto, posicionando con los número de líneas o usando `span` para ocupar más de una celda.

Veamos un ejemplo en el fichero `grid.css` para definir una rejilla de tres columnas. La propiedad `display:grid;` se aplica en el elemento padre de los elementos que queremos disponer en rejilla, que en el caso del ejemplo es un bloque `<div>` con `id="contenedor"`:

```
/* rejilla 3 columnas: líneas de columna de la 1 a la 4 */
#contenedor {
  display: grid;
  grid-template-columns: 20% 50% 30%; /* tres columnas y reparto tamaños columnas */
}
```

Y observe el posicionamiento de los elementos HTML hijos de ese `#contenedor`:

- Se ha movido el `footer` a la primera fila con `grid-row: 1;`
- Se ha modificado el tamaño de `footer` y `header` para que ocupen tres celdas con `grid-column: 1 / span 3` o `grid-column: 1 / 4;`
- Al resto de elementos hijos no se les ha aplicado ninguna propiedad por lo que ocupan una única celda siguiendo el orden en el que aparecen en el contenedor.

```
footer {
  grid-row: 1; /* modifica la fila dónde se posiciona footer */
  grid-column: 1 / span 3; /* equivale a 1 / 4 */
}

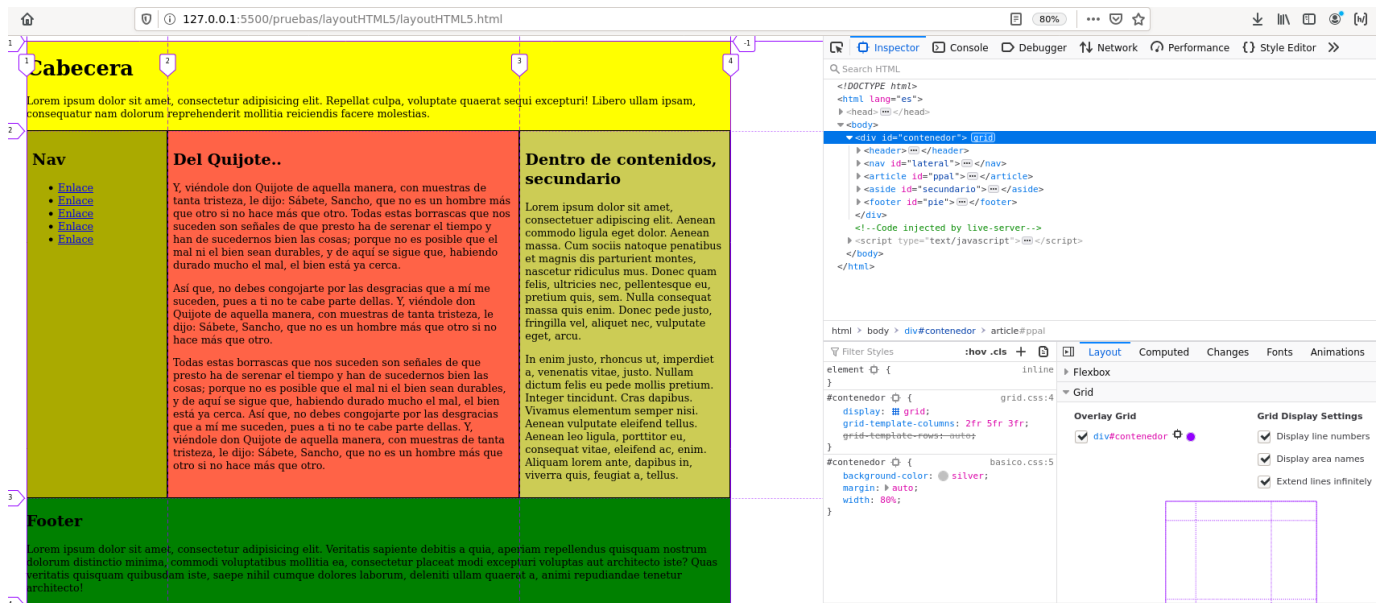
header {
  grid-column: 1 / 4;
```

```
/* equivale a
  grid-column-start: 1;
  grid-column-end: 4; */
}
```

Es posible usar en la plantilla de columnas (`grid-template-columns`) la unidad `fr` (fraction), que simboliza una fracción de espacio total del grid.

```
#contenedor {
  display: grid;
  grid-template-columns: 2fr 5fr 3fr; /* tres columnas y reparto tamaños columnas */
}
```

Puede activar la visualización del grid en el inspector de elementos del navegador, pestaña layout, y ver las celdas, número de líneas, etc. Se puede apreciar en la siguiente figura:



3.1. Grid areas

Se facilita un fichero de ejemplo (`grid_areas.css`) con el uso de la propiedad `grid-template-areas` . Esta propiedad permite **definir una plantilla de la rejilla con nombres simbólicos**. Además hay que asociar esos nombres de áreas a elementos HTML con la propiedad `grid-area` .

```
#contenedor {
  display: grid;
  grid-template-columns: 2fr 5fr 3fr;
  grid-template-areas: /* definimos la plantilla de áreas */
    "cabecera cabecera barra"
    "ads articulo barra";
  "ads footer footer";
}
/* se asocian áreas de la plantilla a elementos HTML */
header {
  grid-area: cabecera;
}
footer {
  grid-area: footer;
}
...
```

Los nombres de área los define el desarrollador y deben ser significativos (en ocasiones se usa como nombre de área el nombre del elemento HTML).

Con grid areas mover los elementos de posición o hacer que ocupen más de una celda es simplemente hacer cambios en la plantilla de `grid-template-areas` (y en `grid-template-columns` si cambia el número de columnas). No es necesario trabajar con los números de fila y columna. Esta forma es menos sensible a errores, y mucho más legible. Observe en el siguiente ejemplo como se modifica el layout simplemente modificando el `template`:

```
#contenedor {
  display: grid;
  grid-template-columns: 2fr 5fr 3fr;
  grid-template-areas: /* definimos BUEVA plantilla de áreas */

```

```

"barra cabecera cabecera "
"barra articulo articulo"
"footer footer ads";
}

```

3.2. Grid anidados

A veces es interesante tener un grid dentro de una de las celdas del grid. Tiene ejemplos de esta situación en el código de la carpeta

`grid_anidados :`

- Ejemplo 01: el elemento nav es una celda del grid *principal*. La lista de enlaces de ese nav se ha definido como grid para disponer sus elementos en horizontal.
- Ejemplo 02: el elemento footer es una celda del grid *principal*. Al mismo tiempo se ha definido el footer como grid para disponer las secciones que hay dentro en columnas.

3.6.4 4. Enlaces de ampliación

- CSS Grid:
 - <https://lenguajecss.com/p/css/propiedades/grid-css>
 - <https://css-tricks.com/snippets/css/complete-guide-grid/>
 - <https://gridbyexample.com/>
 - https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Grid_Layout/Basic_Concepts_of_Grid_Layout
- Flex (o flexbox):
 - https://www.w3schools.com/css/css3_flexbox.asp
 - <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>

3.7 Diseño Adaptativo (RWD)

3.7.1 1. Introducción

Antes de seguir, el significado de RWD: **Responsive Web Design**. El objetivo es adaptar nuestros contenidos y presentación al dispositivo que utilice en cada momento el usuario. Se suele tener en cuenta el ancho de la pantalla, pero también podría considerar la orientación, etc.

Hay tres aproximaciones posibles:

- Mejora progresiva o **"First Mobile"**: iniciar el diseño desde el layout de un móvil e ir "mejorando" hasta llegar a layouts tipo escritorio. La anchura disponible va creciendo. Esta aproximación es la más actual y la que usaremos.
- Degradación progresiva: justo al revés.
- Crear layouts totalmente independientes para cada escenario. Más difícil de mantener

Para usar RWD en nuestra página HTML necesitamos

- Usar una directiva de CSS, las conocidas como *CSS Media Query*.
- Introducir en la cabecera de la página la etiqueta `meta viewport` para que los contenidos funcionen correctamente en móviles, tablets, etc. Esto es algo que ya hacía nuestro editor por defecto.
- Definir los **puntos de ruptura (breakpoints)** en los cuales el layout cambia,

3.7.2 2. Código de ejemplos

Tiene disponible todos los ejemplos de este documento en la Unidad Compartida, carpeta `t12_RWD`. Para ver cómo se adapta el diseño al ancho puede:

- Reducir el ancho de la ventana del navegador
- o quizás más cómodo: abrir el inspector de elementos y jugar con el ancho con la barra que separa la ventana de visualización de la de desarrollo.

3.7.3 3. CSS2 Media Types

Antes de ver las *Media Queries* es frecuente presentar a sus predecesoras, las *Media Types*.

La directiva `@media` de CSS permite aplicar diferentes estilos a diferentes medios, principalmente `screen` y `print`. Si tenemos un fichero de nombre `impresora.css` y añadimos reglas CSS dentro de la directiva `@media print` hacemos que esas reglas CSS sólo se apliquen si la página se va a imprimir:

```
@media print {
  body {
    font-size: 10pt;
  }
  aside {
    display: none;
  }
}
```

Otra posible opción ya comentada en temas previos es indicar el destino (`print|screen|all`) al enlazar el fichero CSS, y todo el fichero se vería afectado por esa restricción:

```
<link rel="stylesheet" media="print" href="impresora.css" />
```

Y en el fichero `impresora.css` no es necesaria la directiva `@media print`, simplemente las reglas CSS:

```
/* impresora_mod.css: No hace falta la directiva @media print:
se enlaza especificando media="print" */
body {
  font-size: 10pt;
}
aside {
  display: none;
}
```

3.7.4 4. CSS3 Media Queries

Las *Media Queries* extienden la idea de las *media types*: en vez de mirar sólo el tipo de dispositivo se pueden consultar otras características del dispositivo: anchura, altura, orientación, etc.

4.1. Sintaxis de una Media Query

Se añade a la sintaxis de una *media type* una o más expresiones, todas las cuales dan como resultado un valor lógico. El resultado es verdadero si el tipo coincide y la expresiones son verdaderas. Si es verdadera, las reglas de estilo incluidas en la directiva *media query* se aplican.

En el siguiente ejemplo las reglas CSS se aplican si:

1. el destino de la página es la pantalla y
2. la anchura disponible es mayor de 480px (de al menos 480px).

```
@media screen and (min-width: 480px) {
  /* Reglas CSS */
}
```

El tipo del media (`screen` en el ejemplo previo) es opcional: si no se incluye su valor por defecto es `all`. Por lo tanto estas dos directivas son equivalentes:

```
@media (orientation: portrait) {
  ...
}

/*es equivalente a */
@media all and (orientation: portrait) {
  ...
}
```

4.2. Ejemplos basicos

Vemos tres ejemplos de uso que además puede probar en los siguientes enlaces

1. Modificar el color de fondo al alcanzar una anchura mínima (`min-width`): https://www.w3schools.com/css/tryit.asp?filename=tryresponsive_mediaquery. El código CSS necesario sería:

```
/* se aplica siempre */
#contenedor {
  background-color: lightblue;
}
/* se aplica si ancho >480px. Entra en colisión con la anterior y se aplica esta última por tener misma especificidad pero aparecer después */
@media screen and (min-width: 480px) {
  #contenedor {
    background-color: lightgreen;
  }
}
```

2. Ahora el cambio de color depende de la orientación del dispositivo (`orientation: landscape`): https://www.w3schools.com/css/tryit.asp?filename=tryresponsive_mediaquery_orientation

```
@media only screen and (orientation: landscape) {
  body {
    background-color: lightblue;
  }
}
```

3. Modificar el layout: Si el ancho es suficiente el menú de navegación pasa a la izquierda del contenido: https://www.w3schools.com/css/tryit.asp?filename=trycss3_media_queries2

```
@media screen and (min-width: 480px) {
  #leftsidebar {
    width: 200px;
    float: left;
  }
}
```

3.7.5 5. Layout CSS y RWD

Nuestro objetivo es modificar el layout de la página para que se adapte al ancho disponible en el dispositivo. Y como se indicaba en la introducción, usaremos un diseño **First Mobile**.

En el diseño **First Mobile** se parte de un diseño apilado (*stacked*) para móvil, y se va añadiendo puntos de ruptura usando *media queries* con `min-width`.

Importante: En cada salto o punto de ruptura

- se mantienen las reglas CSS de los layouts previos
- se pueden añadir nuevas reglas CSS a las del layout previo
- se pueden modificar propiedades del layout previo (y al estar definidas más tarde en el fichero son las que se aplican si hay colisión).

Por eso es importante el orden en el que aparecen las *media queries* (los puntos de ruptura), de menor a mayor.

5.1. Ejemplo 01: RWD con grid y 2 puntos de ruptura

En este ejemplo, disponible en los ficheros `grid-rwd.html` y `grid-rwd.css` fijaremos dos puntos de ruptura, a 580px y 992px. Esto provoca tres layouts distintos:

- uno con ancho hasta 580px.
- otros con ancho mayor que 580px pero menor que 992px
- y el último con ancho mayor que 992px

```
/* grid-rwd.css */
/* layout 01 : No se indica nada, apilado por defecto */

/* layout 02: */
@media (min-width: 580px) {
  #contenedor {
    display: grid;
    grid-template-columns: 1fr 2fr;
  }
  header {
    grid-column: 1 / span 2;
  }
  aside, footer {
    grid-column: 1 / span 2;
  }
}
/* layout 03: tener en cuenta que se aplica
todo lo anterior más lo que se define aquí */
@media (min-width: 992px) {
  #contenedor {
    /* no es necesario especificar display:grid,
    ya especificada en layout 2*/
    grid-template-columns: 1fr 3fr 2fr;
  }
  header, footer {
    grid-column: 1 / span 3;
  }
  aside {
    grid-column: 3;
  }
}
```

En el ejemplo puede observar:

- Hay colisiones en el color de fondo de `header`, y se aplica el del último layout activo.
- En el layout 3 se modifica el número de columnas, y como hay colisión con el previo se aplica el último.
- En el layout 3 no es necesario especificar la propiedad `display: grid`; ya que está activa del layout previo.

5.2. Ejemplo 02: RWD con grid areas

Puede acceder al código en los ficheros:

- `layout_rwd_ga.html` y
- `layout_rwd_ga.css`

En este ejemplo se usan grid con nombres de áreas. En RWD suele ser más sencillo y legible usar áreas, ya que sólo es necesario en cada punto de ruptura modificar la plantilla de áreas (y el número o tamaño de las columnas).

Por ejemplo en el segundo layout del ejemplo tenemos dos columnas y la plantilla:

```
/* Segundo layout: si el ancho es 600 o más..*/
@media screen and (min-width: 600px) {
  #contenedor {
    display: grid;
    grid-template-columns: 1fr 1fr;
    grid-template-areas:
      " footer footer "
      " nav section "
      " aside section "
      " header header ";
  }
}
```

Y en el siguiente layout tres columnas y una plantilla diferente:

```
@media screen and (min-width: 900px) {
  #contenedor {
    grid-template-columns: 1fr 2fr 2fr;
    grid-template-areas:
      " aside header nav "
      " aside section section "
      " aside footer footer ";
  }
}
```

Realizar esa misma distribución usando `grid-column` y `grid-row` es mucho más complejo, propenso a errores, y además menos legible.

5.3. Ejemplos "fuera de temario"

- Ejemplo de uso de RWD con Bootstrap. El código está disponible en `otros/layoutHTML5-BS-RWD.html`.
- Ejemplo práctico de cómo ir mejorando una lista de enlaces con direcciones de correo electrónico a medida que aumenta el tamaño disponible: <https://css-tricks.com/css-media-queries/>.
- Galería de tarjetas: uso de `grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));`. El código está disponible en el fichero `otros/grid-rwd-galeria.html`.

3.7.6 6. Enlaces de consulta adicional

- Qué dice Google sobre RWD: <https://web.dev/responsive-web-design-basics/>.
- Explicando el elemento `meta viewport`: <https://dev.opera.com/articles/an-introduction-to-meta-viewport-and-viewport/>.

3.8 Errores en CSS

- No enlazar el fichero de estilos correctamente en el fichero HTML.
- No seguir las indicaciones del enunciado
- No usar el selector que se indica en el enunciado.

3.8.1 1. Legibilidad

- No formatear el código CSS
- No organizar el código en bloques relacionados o no incluir comentarios que le ayuden a organizar su código

3.8.2 2. Grid

- No usar la propiedad `display: grid` en el elemento adecuado.
- No asociar bien el elemento HTML con el nombre de área (o no hacerlo)
- No elegir nombres de áreas significativos.

3.8.3 3. RWD

- No especificar la unidad en la *media query*: `@media (min-width:600){` en vez de `@media (min-width:600px){`.
- Al hacer un diseño *first mobile* no poner los puntos de ruptura en orden de menor tamaño a mayor tamaño.
- Repetir propiedades **con los mismos valores** en *media query* sucesivos en un diseño *first-mobile*.
- Poner reglas CSS que van dentro de una *media query* fuera de la misma.

3.8.4 4. Selectores

- No usar el selector más adecuado en cada situación.
- O no usar el que implique un código más legible y fácil de mantener.
- Confundir los selectores descendiente, hijo y adyacente.

3.8.5 5. Modelo de caja

- No entender diferencia entre `margin` y `padding`
- No saber usar las imágenes de fondo: repeticiones, posición, etc.
- Solapar la imagen de fondo con el contenido del elemento (cuando no es necesario, claro).
- Al colocar una imagen de fondo no usar un `padding` en función del tamaño de la imagen.

3.8.6 6. Texto

- No saber usar una fuente proporcionada en fichero.
- No usar correctamente las pseudoclases `::after` o `::before`, la propiedad `content:` o el método `attr()`.
- No saber usar `vertical-align`.
- Contadores: no aplicar la regla de los tres pasos:
 - Dónde se inicia o reinicia el contador.
 - Dónde se incrementa.
 - Dónde se muestra.

3.9 Amplia

3.9.1 Bootstrap - Introducción

1. Introducción

Lea esta introducción tomada de *Bootstrap Ya* (<https://www.tutorialesprogramacionya.com/cssya/bootstrapya/detalleconcepto.php?punto=1&codigo=116&inicio=0>)

Bootstrap es un framework (librerías de CSS) que nos facilita y estandariza el desarrollo de sitios web. A partir de la versión 3.x ha sido implementado pensando que se adapte tanto a las pantallas de equipos de escritorio como a móviles y tablets. Bootstrap ha sido desarrollada y es mantenida por la empresa Twitter y la ha liberado como un producto Open Source.

2. Primeros pasos

Acceda a la página *Get started with Bootstrap* <https://getbootstrap.com/docs/5.2/getting-started/introduction/>. Usaremos la última versión estable disponible. Y evitaremos versiones *alpha* o *beta*.

Para empezar bastará enlazar nuestra página HTML inicial con el fichero CDN que nos proponen añadiendo la etiqueta `<link>` en `<head>`

```
...
<link
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/css/bootstrap.min.css"
  rel="stylesheet"
  crossorigin="anonymous"
  integrity="sha384-rbsA2VBKQhggwzxH7pPCaAqO46MgnOM80zW1RWuH61DGLwZJEdK2Kadq2F9CUG65">
<title>Ejemplo Bootstrap</title>
</head>
```

Y en nuestra página web crear un contenedor con un `<h1>` y párrafos de relleno. Observe las clases utilizadas en el bloque `div` o en el elemento `h1`:

```
<body>
<div class="container bg-light pt-3 mt-3">
  <h1 class="text-primary text-center">Ejemplo inicial BS5</h1>
  <p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Libero laudantium commodi eos, officiis sint sunt
    perferendis impedit necessitatibus quasi nisi unde ullam, corrupti dolore et quos quidem dolores temporibus fugit.
  </p>
</div>
</body>
```

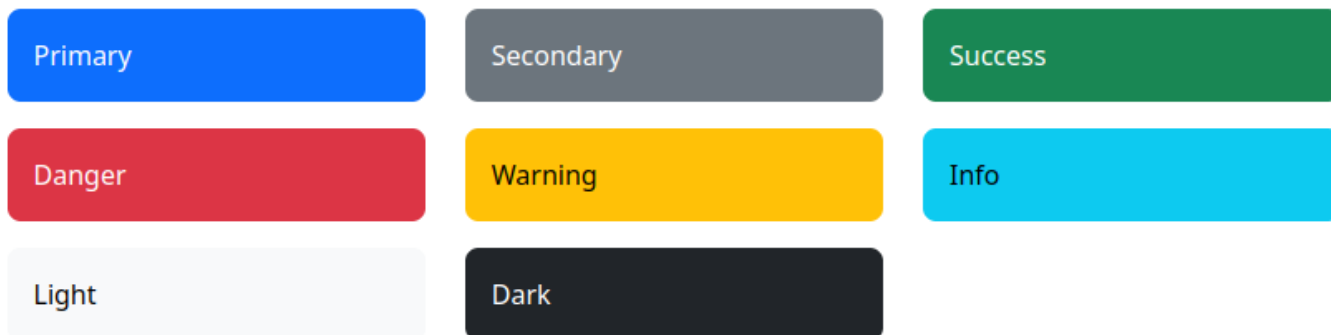
Observe cómo simplemente incluyendo el fichero de estilos CSS de Bootstrap con link y usando clases en los elementos html:

- ha cambiado la tipografía del documento
- el contenedor tiene
 - margen superior (`tp-3`)
 - padding superior (`pt-3`)
 - color de fondo (`bg-light`)
- el `h1`
 - está centrado y
 - tiene color *primario*.

3. Usando clases

3.1. SOBRE COLORES

Bootstrap maneja colores por contextos: *primary*, *secondary*, *success*, *info*, *warning*, *danger*, *light* y *dark*.



Y estos contextos o colores se usa entre otras en las clases:

1. `text-<color>`
2. `bg-<color>`, `text-bg-<color>`

```
<p class="text-primary">This text is important.</p>
<p class="text-success">This text indicates success.</p>
<p class="text-info">This text represents some information.</p>
```

Ejemplos y resumen rápido:

- Color del texto: https://www.w3schools.com/bootstrap5/bootstrap_colors.php
- Color de fondo: https://www.w3schools.com/bootstrap5/bootstrap_colors_bg.php

3.2. PROPIEDADES DE TEXTO

Dispone de clases para alinear el texto:

```
<p class="text-start">Start aligned text on all viewport sizes.</p>
<p class="text-center">Center aligned text on all viewport sizes.</p>
<p class="text-end">End aligned text on all viewport sizes.</p>
```

O para modificar el `font-style`, `font-weight`, `font-size`, etc.

```
<p class="text-uppercase">Uppercased text.</p>
<p class="fs-3">.fs-3 text</p>
<p class="fw-bold">Bold text.</p>
<p class="fst-italic">Italic text.</p>
```

Más detalles en <https://getbootstrap.com/docs/5.2/utilities/text/>

3.3. IMÁGENES

Existen clases para redondear las esquinas de las imágenes, simular miniaturas (*thumbnail*) o redondear la imagen:

- `rounded-circle`
- `rounded`
- `img-thumbnail`

```
<p>The .rounded-circle class shapes the image to a circle:</p>

<p>The .img-thumbnail class creates a thumbnail of the image:</p>

```

Ejemplos de uso en https://www.w3schools.com/bootstrap5/bootstrap_images.php

3.4. TABLAS

Permite poner bordes, alternar color de fondo en filas pares e impares, efecto *hover*, etc.

```
<table class="table table-striped table-hover table-bordered border-primary">
  <thead>
    <tr class="text-bg-primary">
```

```
<th scope="col">#</th>
...
```

Ejemplos de uso en <https://getbootstrap.com/docs/5.2/content/tables/#overview>

3.5. BOTONES

Bootstrap proporciona diferentes estilos de botones:

```
<button type="button" class="btn btn-primary">Primary</button>
<button type="button" class="btn btn-outline-danger">Danger</button>
<a href="#" class="btn btn-success">Link Button</a>
<button type="button" class="btn btn-primary btn-lg">Large</button>
<button type="button" class="btn btn-primary">Default</button>
<button type="button" class="btn btn-primary btn-sm">Small</button>
```

Incluso puede presentar un enlace en forma de botón:

```
<h2>Button Elements</h2>
<a href="#" class="btn btn-success">Link Button</a>
```

Ejemplos en https://www.w3schools.com/bootstrap5/bootstrap_buttons.php

3.6. ICONOS

Para incorporar los iconos que Bootstrap proporciona debe seguir las indicaciones de <https://icons.getbootstrap.com/#install>:

1. Usar un **CDN** con `<link>` al fichero CSS con las definiciones de los iconos

```
<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css">
```

2. Usar **Icon font** e insertar el icono en el código HTML usando el elemento `<i>` sin contenido y aplicándole la clase del icono:

```
<!-- Ejemplo de icono antes de h2 -->
<h2> <i class="bi-alarm"></i> Iconos</h2>
<!-- Ejemplo de icono en botón -->
<button type="button" class="btn btn-primary">
  <i class="bi-arrow-90deg-left"></i> Button
</button>
```

Más información en <https://icons.getbootstrap.com/#icon-font>.

3.7. FORMULARIOS

Tiene detalles y ejemplos de uso de Bootstrap con formularios en este enlace: https://www.w3schools.com/bootstrap5/bootstrap_forms.php

Se le añaden las clases

- `form-label` a las etiquetas.
- `form-control` a los inputs.

Y se pueden a su vez especificar el tipo de input con `class="form-check-input"`, `class="form-select"`, etc.

Además nos pueden interesar posibilidades como:

- Etiquetas *flotantes*: https://www.w3schools.com/bootstrap5/bootstrap_form_floating_labels.php
- Decorar la **validación** de los campos del formulario: https://www.w3schools.com/bootstrap5/bootstrap_form_validation.php

4. Layout

Tiene dos tutoriales a seguir para este apartado:

- Documentación oficial de BS5: <https://getbootstrap.com/docs/5.2/layout/grid/>
- W3schools Grid: https://www.w3schools.com/bootstrap5/bootstrap_grid_system.php

4.1. BOOTSTRAP GRID SYSTEM

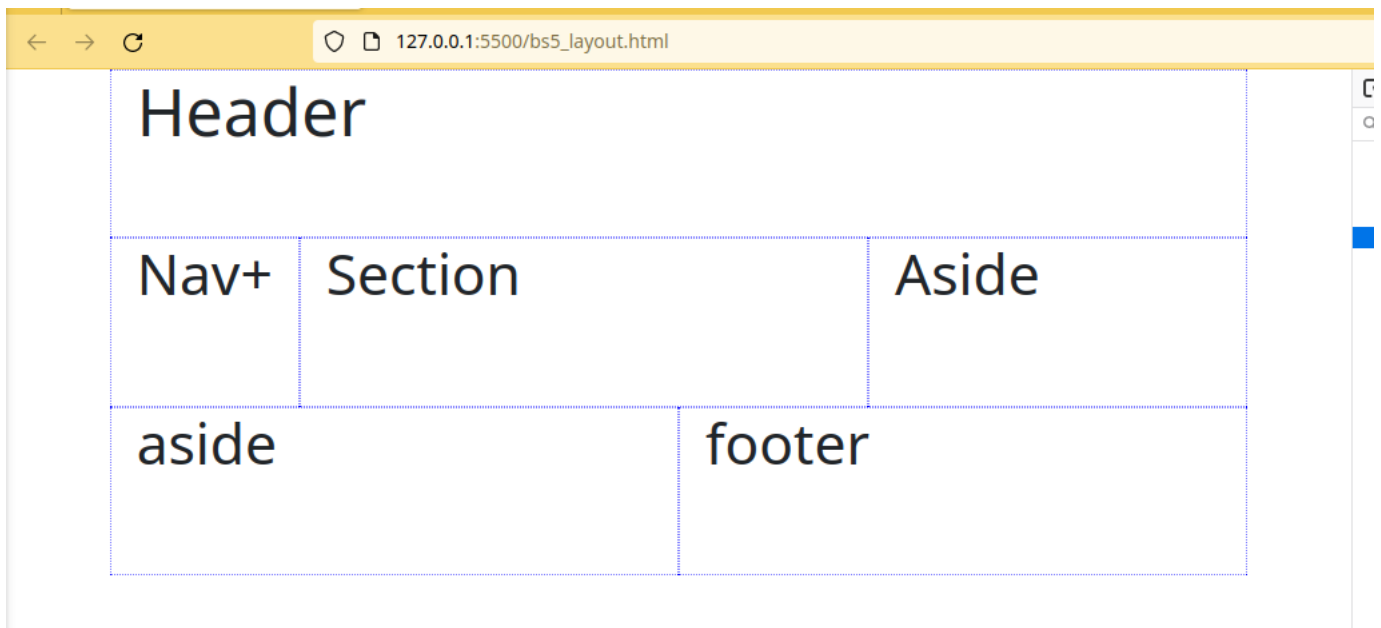
Si hace un layout no adaptativo basta con saber que debe organizar el contenido en filas (`class="row"`), y que cada fila tiene por defecto doce columnas. Para especificar que un elemento es una columna y el tamaño de esa columna se usa la notación `class="col-X"` (siendo X el tamaño sobre 12). Si no se especifican los tamaños se reparte para ocupar toda la fila.

```

21     <div class="container">
22         <div class="row">
23 >         <header class="col"> ...
25         </header>
26     </div>
27     <div class="row">
28 >         <nav class="col-2"> ...
30         </nav>
31 >         <section class="col-6"> ...
33         </section>
34 >         <aside class="col-4"> ...
37         </aside>
38     </div>
39     <div class="row">
40 >         <aside class="col"> ...
42         </aside>
43 >         <footer class="col"> ...
45         </footer>
46     </div>
47 </div>

```

Y el resultado será similar a



En el fichero `layout-BS.html` tiene otro ejemplo de layout.

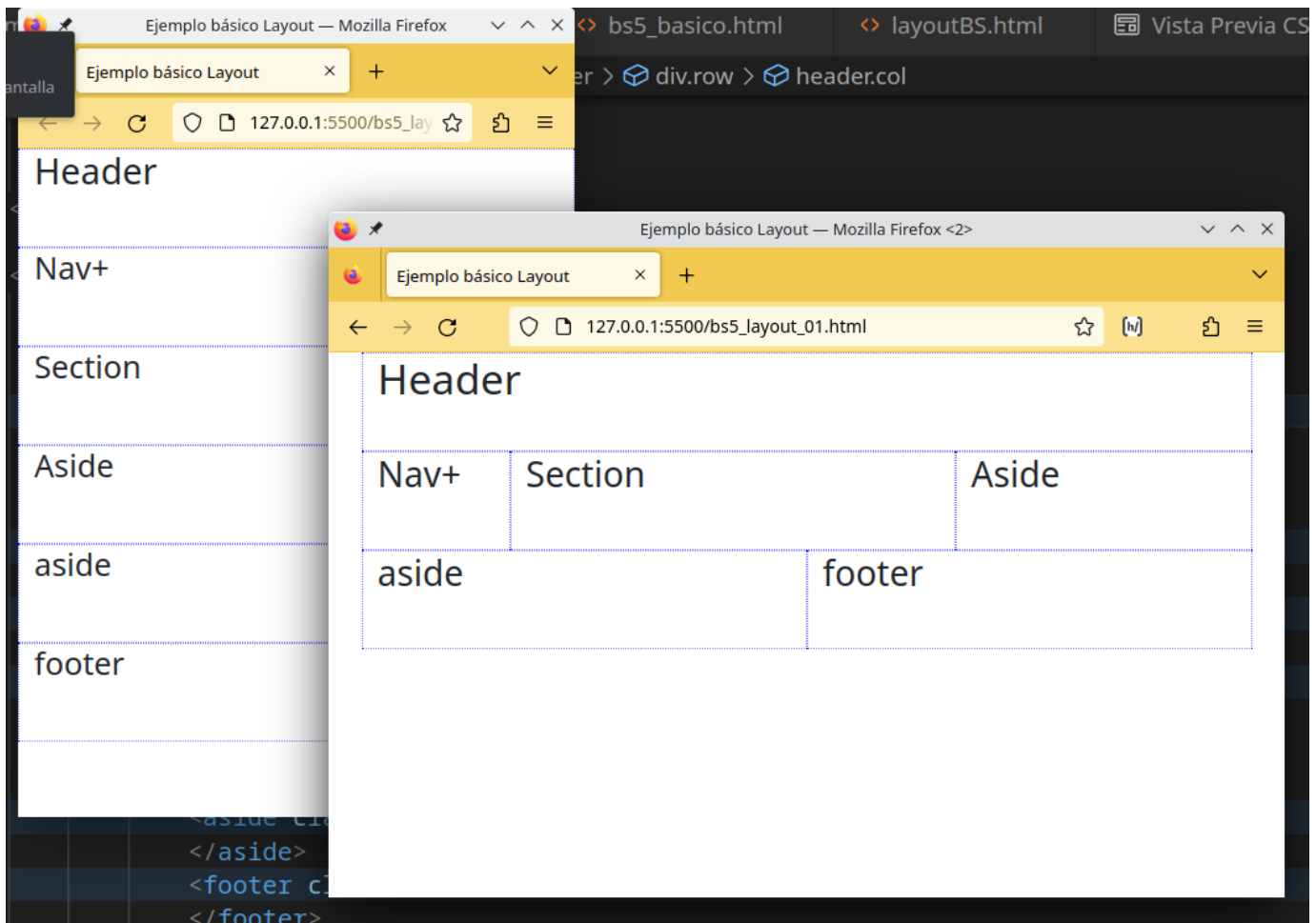
4.2. LAYOUT RWD

BootStrap tiene definidos puntos de ruptura para hacer diseños adaptativos (RWD).

	XSmall	Small	Medium	Large	Extra Large	XXL
Class prefix	<code>.col-</code>	<code>.col-sm-</code>	<code>.col-md-</code>	<code>.col-lg-</code>	<code>.col-xl-</code>	<code>.col-xxl-</code>
Screen width	<code><576px</code>	<code>>=576px</code>	<code>>=768px</code>	<code>>=992px</code>	<code>>=1200px</code>	<code>>=1400px</code>

Si usamos la clase `col-X` se entiende que ese es el layout para todos los anchos de pantalla empezando por el *extra-small*.

Haga una pequeña modificación sobre el ejemplo anterior: sustituir las clases `col-X` por `col-sm-X`. **Ahora para pantallas de ancho mayor a 576px se mantiene el diseño previo pero para menores, al no estar definido el comportamiento, se pasa a un diseño apilado (diseño por defecto).** Vea en la siguiente captura la misma página con dos anchos de pantalla:



Tiene un ejemplo de RWD con bootstrap en el fichero `layoutBS-RWD.html`. En ese ejemplo se usan dos puntos de ruptura y tres layouts distintos.

5. Enlaces principales

- Get started with Bootstrap: <https://getbootstrap.com/docs/5.3/getting-started/introduction/>
- Bootstrap 5 Tutorial W3school: <https://www.w3schools.com/bootstrap5/>

3.9.2 SASS

1. Introducción

¿Qué es Sass?

Sass (short for syntactically awesome style sheets) is a preprocessor scripting language that is interpreted or compiled into Cascading Style Sheets (CSS). SassScript is the scripting language itself.>

Por lo tanto lo que hagamos en Sass debe ser traducido a CSS. Tenemos dos posible sintaxis en Sass:

Sass consists of two syntaxes. The original syntax, called "the indented syntax," uses a syntax similar to Haml.[2] It uses indentation to separate code blocks and newline characters to separate rules. **The newer syntax, SCSS (Sassy CSS), uses block formatting like that of CSS.** It uses braces to denote code blocks and semicolons to separate rules within a block. The indented syntax and SCSS files are traditionally given the extensions .sass and .scss, respectively.

¿Qué añade a CSS? Viene a ofrecer:

- Posibilidad de usar variables
- Usar selectores anidados
- Operadores
- etc.

2. Herramientas

Se utilizará por simplicidad el Plugin de VsCode **Live Sass Compiler** de Glenn Marks. Otra opción más profesional es usar Dart como se muestra en el anexo y que se puede integrar en scripts, herramientas de automatización, etc.

3. Ejemplos de uso

Se proponen alguno de los ejemplos propuestos en la guía de Sass: <https://sass-lang.com/guide>. Tiene el código de los ejemplos disponible en la propia guía (algunos se han modificado para simplificarlos o hacerlos más legibles).

3.1. VARIABLES

Se puede parametriza el uso de medidas ,colores, etc.

Se muestra un ejemplo con el uso de una variable para el color. Cree un fichero de nombre `estilos.scss`

```
$primary-color: green;

p:nth-child(even) {
  color: $primary-color;
}

h1 {
  border-bottom: 2px solid $primary-color;
}
```

El fichero scss no es interpretable por el navegador, es necesario convertirlo antes a CSS. Basta que en VScode use la opción *Paleta de Comandos / Live Sass: Compile Current Sass File*.

Se crean dos archivos: `estilos.css` y `estilos.css.map`. El primero es el que debe incluir en su fichero HTML.

El fichero `estilos.css` obtenido ya no tiene variables:

```
p:nth-child(even) {
  color: green;
}

h1 {
  border-bottom: 2px solid green;
}
```

```

}
/** sourceMappingURL=estilos.css.map */

```

Nota: si desea puede activar el modo **watch** en el plugin (*Live Sass: Watch mode*) que detecta cambios en los ficheros .sass y automáticamente hace la compilación a CSS. Para detenerlo *Live Sass: Stop watching*

3.2. SELECTORES ANIDADOS

Ejemplo de una barra de navegación y se modifica para anidar los selectores:

```

/* Anidación */
nav {
  ul {
    padding: 0;
    list-style: none;
    border: 1px solid $primary-color;
    display: grid;
    grid-template-columns: repeat(5, 1fr);

    li {
      border: 1px solid $primary-color;
      text-align: center;

      a {
        display: block;
        padding: 6px;
        text-decoration: none;
      }
    }
  }
}

```

El resultado traducido sería:

```

/* Anidación */
nav ul {
  padding: 0;
  list-style: none;
  border: 1px solid green;
  display: grid;
  grid-template-columns: repeat(5, 1fr);
}

nav ul li {
  border: 1px solid green;
  text-align: center;
}

nav ul li a {
  display: block;
  padding: 6px;
  text-decoration: none;
}

```

3.3. MIXINS

A mixin lets you make groups of CSS declarations that you want to reuse throughout your site. It helps keep your Sass very DRY. You can even pass in values to make your mixin more flexible. Here's an example for theme

To create a mixin you use the `@mixin` directive and give it a name. We've named our mixin `theme`. We're also using the variable `$theme` inside the parentheses so we can pass in a theme of whatever we want. After you create your mixin, you can then use it as a CSS declaration starting with `@include` followed by the name of the mixin.

Cree un *mixin* de nombre `theme` con un parámetro de nombre `color` (que tiene como valor por defecto `silver`):

```

/* mixin con variable */
@mixin theme($color: silver) {
  background: $color;
  box-shadow: 0 0 1px rgba($color, .25);
  color: #fff;
}

```

Para utilizarlo en ese mismo fichero se usa la directiva `@include` con el nombre del mixin.

```

.info {
  @include theme; /* usa tema por defecto, silver */
}

.alert {
  @include theme($color: red);
}

```

```
.success {
  @include theme($color: #00dd00);
}
```

4. Enlaces

- Sass: <https://sass-lang.com/>
- Sass Guide: <https://sass-lang.com/guide>
- VScode plugin: [Live Sass Compiler](#) de Glenn Marks: Compile Sass or Scss to CSS at realtime.
- Código dart: <https://github.com/sass/dart-sass/releases>
- W3schools Sass: <https://www.w3schools.com/sass/>

5. Anexos

5.1. DART

Puede descargar `dart-sass` para Linux, descomprimir en el directorio `/opt` y añadir esa ruta al `PATH`. Puede seguir las instrucciones de estos enlaces:

Install Anywhere (Standalone)

You can install Sass on Windows, Mac, or Linux by downloading the package for your operating system from [GitHub](#) and [adding it to your PATH](#). That's all—there are no external dependencies and nothing else you need to install.

PATH:

- Open the `.bashrc` file in your home directory (for example, `/home/your-user-name/.bashrc`) in a text editor.
- Add `export PATH="your-dir:$PATH"` to the last line of the file, where `your-dir` is the directory you want to add.
- Save the `.bashrc` file.
- Restart your terminal.

Tras descomprimir en `/opt` y modificar el `PATH`:

```
alu@debian11:~$ ls -l /opt/dart-sass/
total 8
-rwxr-xr-x 1 root root 592 feb 19 10:23 sass
drwxr-xr-x 2 root root 4096 feb 19 10:23 src
alu@debian11:~$ echo $PATH
/opt/dart-sass:/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games
alu@debian11:~$
alu@debian11:~$ sass --version
1.58.3
alu@debian11:~$
```

5.1.1. Utilización

El fichero `scss` no es interpretable en el navegador, es necesario convertirlo antes a `CSS`.

```
alu@debian11:~/bloque_CSS/t20_sass$ sass estilos.scss estilos.css
alu@debian11:~/bloque_CSS/t20_sass$
```

La segunda opción es que Sass traduzca cuando detecte cambios en el fichero con la opción `--watch`:

```
alu@debian11:~/bloque_CSS/t20_sass$ sass --watch estilos.scss estilos.css
Sass is watching for changes. Press Ctrl-C to stop.

[2023-02-19 10:47:02] Compiled estilos.scss to estilos.css.
```

También es posible aplicar la traducción sobre directorios completos:

You can watch and output to directories by using folder paths as your input and output, and separating them with a colon. In this example:

```
sass --watch app/sass:public/stylesheets
```

Sass would watch all files in the `app/sass` folder for changes, and compile CSS to the `public/stylesheets` folder.

3.9.3 Linter CSS

1. Introducción

Lint es una herramienta de programación; originalmente lint era el nombre de una herramienta de programación utilizada para detectar código sospechoso, confuso o incompatible entre distintas arquitecturas en programas escritos en C; es decir, errores de programación que escapan al habitual análisis sintáctico que hace el compilador.

Actualmente, lint se utiliza para referirse a las herramientas que analizan programas o documentos en busca de errores sintácticos, de posibles fuentes de errores, o incluso programas que no siguen reglas de estilo.

Es posible que el editor que utilice traiga ya integradas algunas capacidades para detectar errores CSS, indicando por ejemplo propiedades desconocidas o reglas CSS vacías (sin propiedades).

Pero en general además se utilizan herramientas adicionales que amplían esas capacidades como por ejemplo stylelint.

2. stylelint

stylelint es un linter CSS que se basa en `nodejs` y su gestor de paquetes `npm`. La documentación está disponible en <https://stylelint.io/>

Por lo tanto debe tener instalados node y npm. Si no:

```
sudo apt update
sudo apt install node npm
```

Una vez instalados:

1. Inicializar el proyecto: Abra la terminal y escriba:

```
npm init -y
```

2. Instalar el motor Stylelint y la configuración estándar: escriba en la terminal:

```
npm install --save-dev stylelint stylelint-config-standard
```

3. Crear el archivo `.stylelintrc.json` y añadir este contenido:

```
{
  "extends": "stylelint-config-standard",
  "rules": {
  }
}
```

Aviso: si no tiene instalada la última versión de node es posible que la versión de stylelint que se instala por defecto no sea compatible y tendría que instalar versiones inferiores que sean compatibles. Por ejemplo si tiene instalado node versión 18 tendría que ejecutar los siguientes comandos con versiones previas de stylelint:

```
rm -rf node_modules package-lock.json
npm install --save-dev stylelint@16.2.0
npm install --save-dev stylelint-config-standard@36.0.0
```

Con esta configuración estándar por ejemplo se comprueba que:

- se incluye una familia genérica con `font-family`
- los selectores se escriben en minúsculas
- no hay reglas CSS vacías
- no hay propiedades que no existen
- no hay selectores desconocidos
- etc.

2.1. FICHERO CONFIGURACIÓN BÁSICO

La configuración básica del linter se almacena en el fichero `.stylelintrc.json`. El fichero debe residir en el directorio del proyecto.

Su contenido inicial podría ser:

```
{
  "extends": "stylelint-config-standard",
  "rules": {
    "color-named": "never",
    "selector-class-pattern": null,
    "number-max-precision": 2
  }
}
```

Observe que en la sección de reglas:

- Se ha **añadido** una regla que no está activa por defecto en la configuración estándar que prohíbe los nombres de colores.
- Se ha **desactivado** una regla asignándole el valor null. En este ejemplo la configuración estándar es muy estricta con los nombres de las clases CSS, exige el formato "kebab-case" (ej: `.mi-boton`). Si escribe `.miBoton` (camelCase) o `.mi_boton` (snake_case) le daría error.
- Se ha **modificado** una regla existente: a veces no desea activar ni desactivar, sino modificar p poner un límite. En este caso se usa `number-max-precision` para limitar los decimales a 2.

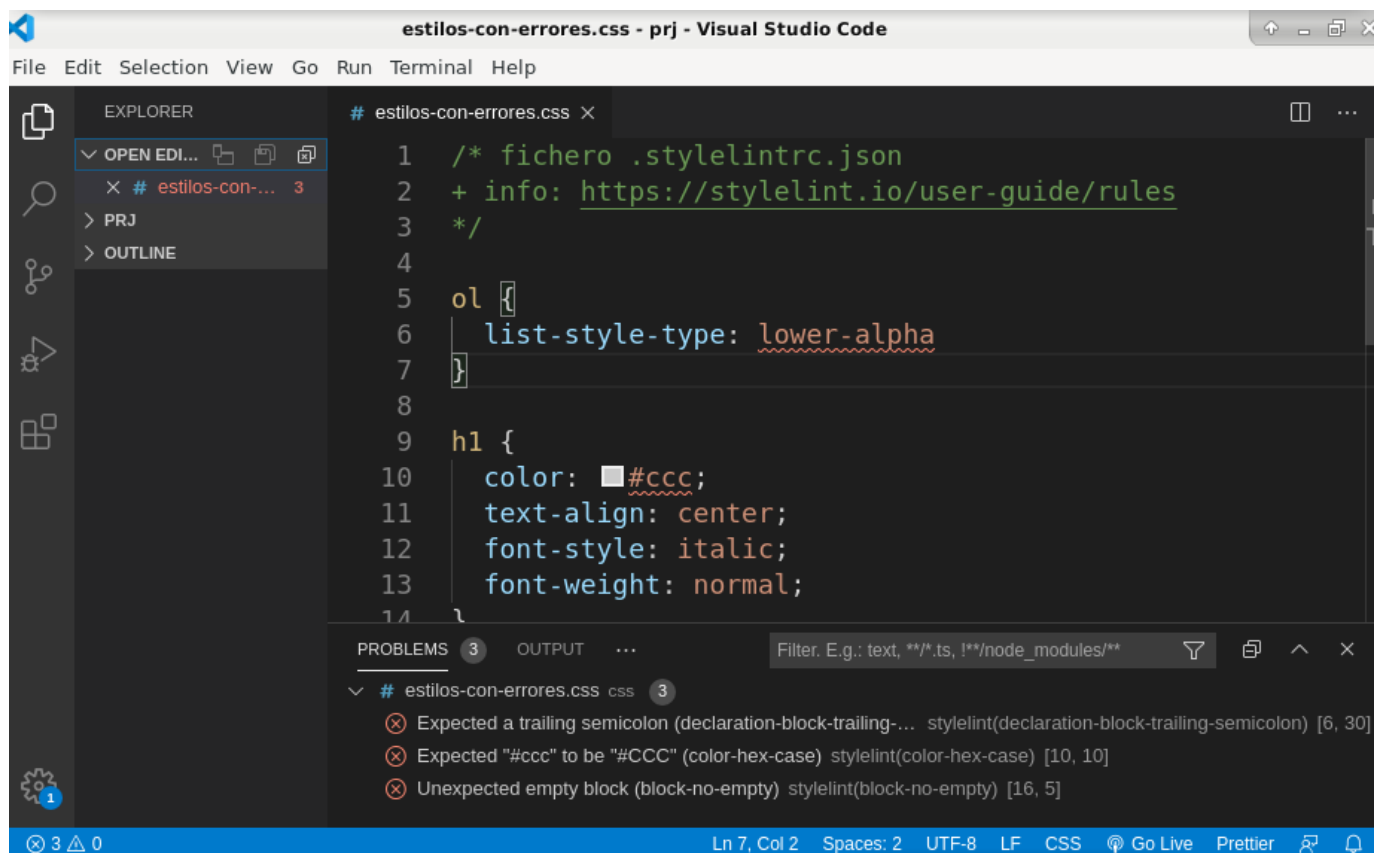
2.2. USANDO UN EDITOR

Para usar desde un editor (vscode o similares) es posible que necesite instalar algún plugin adicional.

El plugin para VSCode es [stylelint](#). Una vez instalado el editor marca el error:

- en el propio código (con subrayados en rojo)
- en la consola de errores.

En la siguiente figura puede observar como en las líneas 4 y 10 se marcan errores y cómo esos errores se muestran en la ventana de "Problems"



2.3. USO DESDE LA LÍNEA DE COMANDOS

Para usar `stylelint` desde la línea de comandos basta indicarle la ruta al ejecutable y la ruta a los ficheros a revisar. Como la instalación ha sido local (`--save-dev`)

```
$ ./node_modules/.bin/stylelint estilos/estilos-con-errores.css

estilos/estilos-con-errores.css
 6:1  ✖ Expected "H3" to be "h3"          selector-type-case
 7:36 ✖ Expected "#aabbcc" to be "#abc"      color-hex-length
11:3  ✖ Unexpected unknown property "text-alig" property-no-unknown
14:9  ✖ Unexpected empty block               block-no-empty
18:1  ✖ Unexpected unknown type selector "boton" selector-type-no-unknown
19:13 ✖ Unexpected unit                      length-zero-no-unit

6 problems (6 errors, 0 warnings)

$
```

Como resultado nos indica para cada fichero revisado:

- fila y columna
- explicación del error
- y la regla (rule) que genera ese aviso.

Este último campo, el nombre de la regla, nos puede servir para modificar nuestro fichero de configuración o buscar ayuda sobre ese error.

3. Enlaces

- Node: <https://nodejs.org/es/>
- Stylelint: <https://stylelint.io/>
- Módulo Configuración estándar: <https://github.com/stylelint/stylelint-config-standard>
- Reglas y ejemplos: <https://stylelint.io/user-guide/rules/list>
- Plugin Vscode: <https://marketplace.visualstudio.com/items?itemName=stylelint.vscode-stylelint>

4. Anexos

4.1. DESACTIVANDO REGLAS EN EL CÓDIGO CSS

En ocasiones nos puede interesar que el linter no revise algún bloque del código o alguna propiedad CSS. Tiene más información en <https://stylelint.io/user-guide/ignore-code>

4.1.1. Desactivando el linter en un bloque de código

Con un comentario especial (`stylelint-disable` o `stylelint-enable`) le indicamos al linter que desabilite/habilite la revisión:

```
/* stylelint-disable */
nav ul {
  border: 1px red solid;
  background-color: beige;
  list-style-type: none;
  padding-left: 0;
}
/* stylelint-enable */
```

4.1.2. Desactivando alguna regla en un bloque de código

Similar al apartado anterior, pero se añade en el comentario una lista con las reglas que quiere desactivar en ese bloque:

```
/* stylelint-disable no-duplicate-selectors */
nav ul {
  border: 1px red solid;
  background-color: beige;
  list-style-type: none;
  padding-left: 0;
}

nav ul {
  display: grid;
  grid-template-columns: repeat(5, 1fr);
}
```

```
}
/* stylelint-enable no-duplicate-selectors */
```

4.2. INSTALACIÓN STYLELINT EN WINDOWS 10

1. Instalar Node (incluye npm): <https://nodejs.org/es/> con opciones por defecto.
2. Comprobar node y npm

```
C:\Users\IEUser> node -v
v18.13.0

C:\Users\IEUser> npm -v
8.19.3

C:\Users\IEUser>
```

3. Instalar stylelint (desde npm)

```
C:\Users\IEUser> npm install -g stylelint

added 172 packages, and audited 173 packages in 11s

27 packages are looking for funding
run `npm fund` for details

found 0 vulnerabilities
npm notice
npm notice New major version of npm available! 8.19.3 -> 9.3.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v9.3.1
npm notice Run npm install -g npm@9.3.1 to update!
npm notice

C:\Users\IEUser> stylelint --version
14.16.1

C:\Users\IEUser>
```

4. Instalar stylint.config-standard (desde npm)

```
C:\Users\IEUser> npm install -g stylelint-config-standard

added 174 packages, and audited 175 packages in 7s

27 packages are looking for funding
run `npm fund` for details

found 0 vulnerabilities

C:\Users\IEUser>
```

5. Crear fichero configuración en directorio del proyecto

```
C:\Users\IEUser\Documents\prj>ls -l
-rwxrwxr-x+ 1 IEUser      None   48 Jan 20 18:52 .stylelintrc.json
-rwxrwxr-x+ 1 IEUser      None   81 Jan 20 18:54 estilos.css
-rwxrwxr-x+ 1 IEUser      None 1090 Jan 20 18:47 index.html

C:\Users\IEUser\Documents\prj>type .stylelintrc.json
{
  "extends": "stylelint-config-standard"
}
C:\Users\IEUser\Documents\prj>
```

6. Probar en línea de comandos

```
C:\Users\IEUser\Documents\prj>stylelint estilos.css

estilos.css
1:3  ✖ Expected indentation of 0 spaces      indentation
3:5  ✖ Expected "20px 20px" to be "20px"     shorthand-property-no-redundant-values
3:5  ✖ Expected indentation of 2 spaces      indentation
8:1  ✖ Unexpected missing end-of-source newline no-missing-end-of-source-newline

4 problems (4 errors, 0 warnings)

C:\Users\IEUser\Documents\prj>
```

7. Plugin en Vscode: <https://marketplace.visualstudio.com/items?itemName=stylelint.vscode-stylelint>

4. 03. JavaScript

4.1 Introducción JavaScript 01

4.1.1 1. Referencias

- JavaScript Tutorial W3Schools: <https://www.w3schools.com/js/default.asp>
- Introducción a JavaScript en Uniwebsidad ahora en archive.org
- Eloquent JavaScript: <https://eloquentjs-es.thedoho.mx/>
- Manual JavaScript : <https://desarrolloweb.com/home/javascript>

4.1.2 2. Entorno de trabajo

- En este bloque se usa una máquina virtual con Linux y VSCode como editor.
- Es posible que se añadan a VSCode algunos plugins para JavaScript.

4.1.3 3. Introducción

Sobre la historia de JavaScript y ECMAScript puede ver el apartado **Historia** de esta página: <https://desarrolloweb.com/home/javascript>

Y para tener una primera aproximación al uso de JavaScript puede ver los ejemplos de esta página https://www.w3schools.com/js/js_intro.asp. Entre ellos JavaScript le permite:

- reaccionar a eventos.
- modificar el contenido de la página HTML: cambiar el contenido de un elemento, añadir nuevos elementos o borrar elementos.
- modificar los estilos CSS que se aplican.
- verificar valores de entradas en formularios.
- o hacer cálculos, presentar resultados, etc,

4.1.4 4. Sintaxis básica JavaScript

4.1. Cómo incluir el código

Recuerde que el código JavaScript se interpreta y ejecuta en el navegador. Y para ello debe conocer cómo incluir el código javascript en la página HTML.

Puede incluir el código en el head y en el body. Y puede hacerlo incluyendo directamente el código o incluyendo un enlace a un fichero con el código javascript.

Tiene los detalles es estos enlaces:

- [W3schools: JavaScript Where To](#)
- [Uniwebsidad: Cómo incluir JavaScript en documentos XHTML](#)

Tiene ejemplos de cómo hacerlo en los ficheros `ejemplo001.html` y `ejemplo002.html`. Un extracto de los mismos:

ejemplo01.html

```
<h1>Ejemplos inclusión código js</h1>
<script>
  // comentario en script JS
  alert("Hola desde body fichero HTML");
</script>
<script src="ejemplo001.js"></script>
```

O un ejemplo de fichero con funciones en `head`


```
...
<title>Ejemplo 02</title>
<script src="ejemplo002.js"></script>
</head>
<body>
```

Tenga en cuenta que el lugar donde incluya el código puede ser importante ya que si se ejecuta código que tiene que acceder a elementos HTML y estos todavía no se han cargado el resultado no es el esperado.

4.2. Comentarios JavaScript

Los comentarios en JavaScript pueden ser de dos tipos:

- comentarios de línea
- comentarios de bloque

```
let media = 0; // media de todo el grupo

/* Pendiente aclarar el cálculo de las medias
por trimestres o anuales */
```

4.3. Variables y sus tipos

Para una introducción al uso de variables y sus tipos debe consultar:

- Apartados de W3Schools
- [JavaScript Variables](#)
- [JavaScript Data Types](#)
- o en Uniwebsidad (web.archive.org):
- [3.1 Variables](#)
- [3.2. Tipos de variables](#)

Resumiendo, se definen las variables con `let` (evitar el uso de `var`). Si no se deben modificar usar `const`. Procurar definir las agrupadas al inicio del script o del bloque. Y para ver su tipo se puede usar `typeof`

Tipos más frecuentes:

- numéricas (Number)
- cadenas de texto (String)
- lógicas o booleanas (Boolean) que toman los valores `true` y `false`
- Y los [tipos especiales](#):
- `undefined`: es el tipo de datos de una variable que no ha sido inicializada, no se le ha asignado ningún valor
- `null`: es un valor especial que asigna el programador e indica que la variable tiene un valor desconocido o vacío.

Ejemplos de uso:

```
ejemplo003.js

// Numbers:
let length = 16;
let weight = 7.5;

console.log(typeof(weight))

// Strings:
let color = "Yellow";
let lastName = "Johnson";

// Booleans
let x = true;
let y = false;

// undefined
let z;
console.log("z: ", z)
console.log(typeof z)
```

```
// null
let w = null;
console.log("w: ", w)
console.log(typeof w) // object
```

Más adelante se presentarán otros tipos:

- arrays
- objetos
- fechas

Tiene el código de estos ejemplos en `ejemplo003.html` y `ejemplo003.js`.

4.4. Operadores

Consultar:

- [W3Schools: JavaScript Operators](#)
- [Uniwebsidad: 3.3. Operadores](#)

4.4.1. OPERADOR DE ASIGNACIÓN

Este operador se utiliza para guardar un valor específico en una variable. También se expresa indicando que se crea un "vinculación" entre la variable y el valor asignado:

```
let edadA = 7;
let edadB = 8;
let media = 0;
```

Después de asignar se puede usar el valor de la variable en expresiones o modificar el valor de la asignación:

```
let suma = edadA + edadB;
media = suma/2;
```

Si se encuentra una asignación como `variable = variable + expresión;` se puede expresar también usando el operador `+=` y quedaría `variable += expresión;`

```
let edadC = 24;

// a) forma "normal"
suma = suma + edadC;

// b) forma compacta
suma += edadC;
```

Se puede usar esta notación con otros operadores como la resta, producto, etc. Tiene estos ejemplos ampliados en `ejemplo004.html` y `ejemplo004.js`.

4.4.2. OPERADORES MATEMÁTICOS

- `+` Suma
- `-` Resta
- `*` Multiplica
- `**` Potencia
- `/` Divide
- `%` Módulo (resto)
- `++` Incrementa
- `--` Decrementa

4.4.3. OPERADORES LÓGICOS

- `&&` Y lógico (AND)
- `||` O lógico (OR)

- **!** NO lógico (NOT)

4.4.4. OPERADORES RELACIONALES

Los más habituales:

- **==** compara si los valores son iguales
- **!=** distinto que
- **>** mayor que
- **<** menor que
- **>=** mayor o igual que
- **<=** menor o igual que

También existe el operador ternario **?** que asigna un valor entre dos posibilidades al evaluar una condición:

```
// let variable = (condición) ? valor1 : valor2;
let puedeJugar = (edad < 18) ? false : true;
```

Puede ocurrir que al hacer la comparación los valores sean iguales pero no los tipos. Por esa razón existe estos dos operadores:

- **===** compara si los valores *y los tipos* son iguales
- **!==** si son distintos los valores o son distintos los tipos

4.4.5. OPERADORES DE STRING

- **+** concatena dos cadenas
- **+=** concatena y asigna

Tenga en cuenta que si añade un número a un string el resultado será de tipo string:

```
let a = 5;
let b = "6";
let suma = a + b;
// => suma = "56"
```

4.5. Mostrando resultados

Hasta ahora se han mostrado los resultados con la función `console.log()`. Pero tenemos más posibilidades.

En este enlace se nos muestran otras posibilidades: https://www.w3schools.com/js/js_output.asp

- Escribir en la salida HTML usando `document.write()`.
- Escribir en una caja de alerta usando `window.alert()`.
- Escribir en la consola del navegador usando `console.log()`.

Existe otra posibilidad: mostrar los resultados en el documento HTML usando el modelo DOM y sus métodos.

```
<p id="demo"></p> <!-- para resultado -->
<script>
  let edadA = 23;
  let edadB = 26;
  let res = (edadA + edadB)/2;

  // modifica el contenido del elemento con id="demo"
  document.getElementById("demo").innerHTML = res;
</script>
```

Tiene otro ejemplo más completo en `ejemplo009.html` y `ejemplo009.js`.

Nota: Using `document.write()` after an HTML document is loaded, will delete all existing HTML:

4.1.5 5. Entrada de datos: confirm y prompt

Tiene ejemplos de su uso en este enlace: [JavaScript Popup Boxes](#)

En resumen:

- `confirm` devuelve `true` o `false` y permite bifurcar el código en función de lo que pulse el usuario.
- `prompt` solicita un valor al usuario. Devuelve:
 - el valor introducido si el usuario pulsa *OK*.
 - `null` si el usuario pulsa *Cancel*.

Tenga en cuenta que `prompt` devuelve el valor introducido como una cadena de caracteres. Tiene ejemplos de su uso en `ejemplo010.html` y `ejemplo010.js`.

4.2 Introducción JavaScript 02

4.2.1 1. Estructuras de control

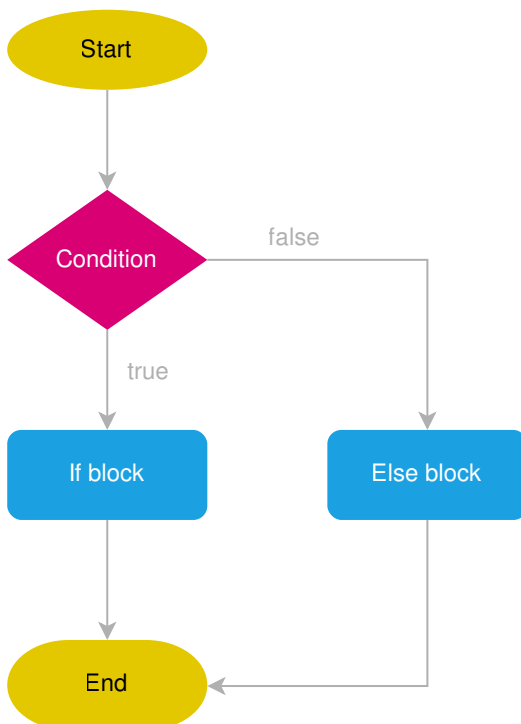
Las imágenes de estructuras de control de este tema están tomadas de este tutorial: [JavaScript Tutorial](#)

1.1. Selectivas

Puede consultar estos enlaces:

- [JavaScript if, else, and else if](#)
- [JavaScript Switch Statement](#)

1.1.1. SELECTIVA IF-ELSE



Ejemplo

```

let edad = Number(prompt("Edad: "));

if (edad == null || edad < 18) {
  alert("Menor de edad o no la indica");
}
else {
  alert("Mayor de edad");
}
  
```

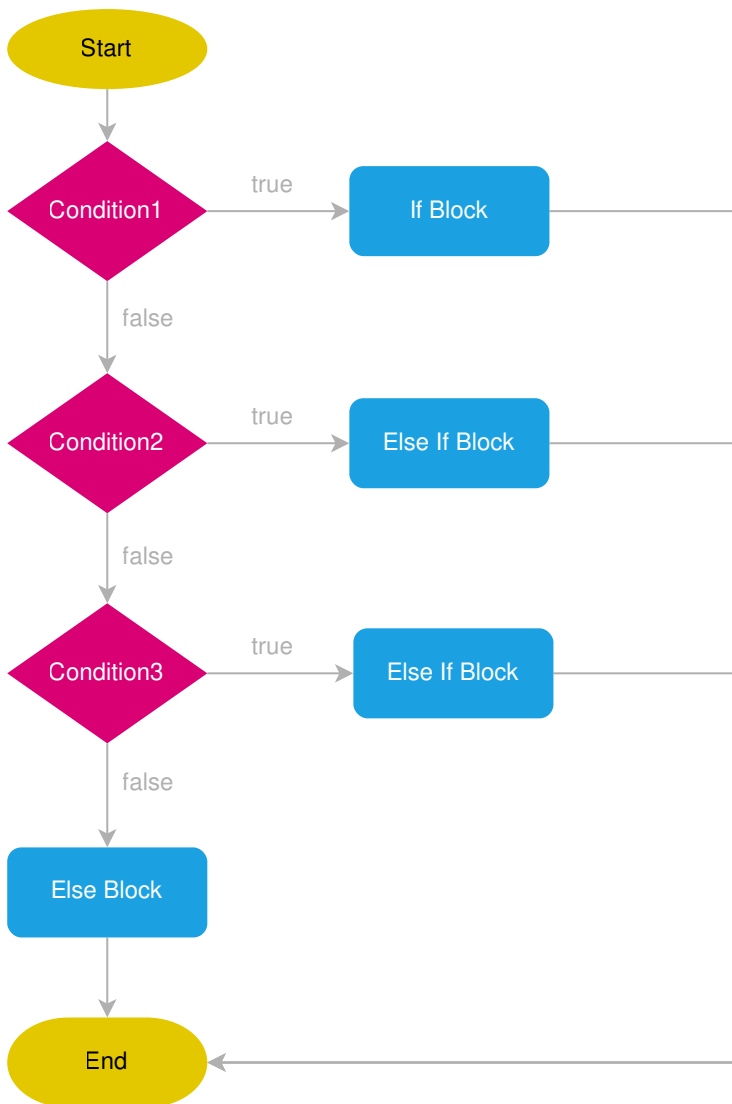
1.1.2. SELECTIVA IF-ELSEIF

Se usa `else if` para especificar una nueva condición si la condición inicial es falsa. Por ejemplo

```

if (time < 10) {
  greeting = "Good morning";
} else if (time < 20) {
  greeting = "Good day";
} else {
  greeting = "Good evening";
}
  
```

Como se observa en esta figura se pueden "encadenar" varios `else if`:



Ejemplo de varios `else if` encadenados:

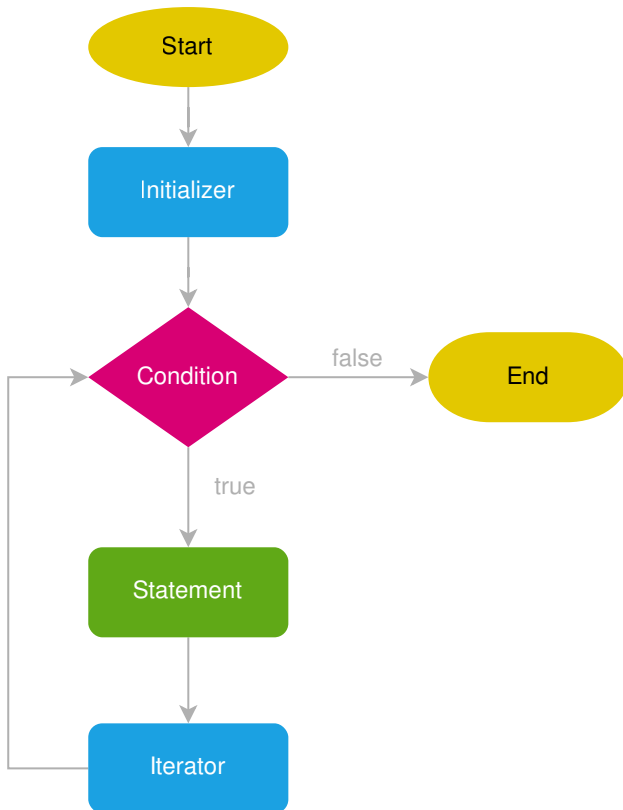
```

let nota = Number(prompt("Nota: "));

if (nota < 5) {
  alert("Suspendo");
} else if (nota < 6) {
  alert("Aprobado");
} else if (nota < 9) {
  alert("Notable");
} else if (nota <= 10) {
  alert("Sobresaliente");
} else {
  alert("Nota fuera de rango");
}
  
```

1.2. Bucles: bucle for

Vea el diagrama clásico de un bucle `for`



[Univebsidad]"mientras la condición indicada se siga cumpliendo, repite la ejecución de las instrucciones definidas dentro del for. Además, después de cada repetición, actualiza el valor de las variables que se utilizan en la condición".

- La "inicialización" es la zona en la que se establece los valores iniciales de las variables que controlan la repetición.
- La "condición" es el único elemento que decide si continua o se detiene la repetición.
- La "actualización" es el nuevo valor que se asigna después de cada repetición a las variables que controlan la repetición.

En el código de ejemplo vea que el paréntesis tiene tres partes separadas: inicialización, condición y actualización:

```

for (let i=1; i<=10; i++){
  console.log(i);
}

let suma = 0;
for (let i=1; i<=10; i++){
  suma = suma + i;
}
console.log(suma);
  
```

Hay ejemplos completos en los ficheros `ejemplo016.html` y `ejemplo16.js`.

4.2.2.2. Arrays

Un array almacena múltiples valores sobre un nombre único. Se puede acceder a un valor concreto usando el nombre del array y un índice (su posición en el array).

Existen varias formas de definir un array y la más habitual es la notación "literal" usando corchetes y una lista de valores separados por comas. Es posible conocer el tamaño del array con la propiedad `length`:

```

let marcas = ["Volvo", "Toyota", "Seat" ];
console.log(marcas.length)
  
```

Para acceder al valor de un elemento (o modificarlo) se usa su posición **teniendo en cuenta que se indexa a partir del valor cero**:

```

console.log(marcas[0])
console.log(marcas[1])
  
```

```
marcas[1] = "Renault";
console.log(marcas[1])
```

2.1. Arrays y bucles

Los bucles `for` son de gran utilidad al trabajar con arrays ya que se puede recorrer el array usando la variable de control del bucle como índice:

```
let temperaturas = [12.5, 13.6, 8.0, 19.2, 23.4];
for (let i = 0; i < temperaturas.length; i++) {
  console.log(i, " - ", temperaturas[i]);
}
```

En el caso de que no sea necesario usar la variable de control en el cuerpo del bucle puede usar la "variante" `for of`

```
let media = 0;
let suma = 0;

for (let temp of temperaturas) {
  suma += temp
}

media = suma/temperaturas.length;
console.log("Media: ", media)
```

El bucle `for of` es equivalente a este

```
for (let i = 0; i < temperaturas.length; i++) {
  suma += temperaturas[i];
}
```

Hay ejemplos completos en los ficheros `ejemplo020.html` y `ejemplo020.js`.

4.2.3 3. Métodos predefinidos

3.1. Cadenas

Puede consultar este enlace: https://www.w3schools.com/js/js_string_methods.asp

Un string o cadena de caracteres:

- dispone de la propiedad `length`
- se puede acceder a cada carácter con la notación de corchetes.
- se puede recorrer con un bucle `for`.

```
let nombre = "Ana";
let apellido = "Herrera";

console.log(nombre[0]);
console.log(nombre.length);

for (let letra of apellido)
  console.log(letra)
```

Además dispone de varios métodos predefinidos, entre ellos:

- Concatenar con `+` o con el método `concat()`.
- `toUpperCase()`, `toLowerCase()`, `trim()`: mayúsculas, minúsculas, eliminar espacios
- `indexOf(character)`: devuelve la posición de `character` en el string
- `slice(ini, fin)`: extrae desde `ini` hasta `fin` y devuelve el resultado.
- `replace(orig, dest)`: reemplaza la cadena `orig` con la cadena `dest` y devuelve el resultado.
- `split(separador)`: a partir de un string devuelve como resultado un array usando como delimitador la cadena `separador`

Vea un ejemplo de uso de `split()`:

```
let lista = empresas.split(",");
console.log("empresas: ", empresas);
console.log("  empresas.length: ", empresas.length);
```



```
console.log("lista: ", lista);
console.log("  lista.length: ", lista.length);
```

Hay ejemplos completos en los ficheros `ejemplo025.html` y `ejemplo025.js`.

3.2. Números

Entre otras funciones puede usar estas:

- `isNaN()` : devuelve verdadero si el valor pasado no es un numero (*Not A Number*)
- `toFixed(N)` : devuelve un string con `N` decimales.
- `Number()` : convierte string a `Number`.

Ejemplo:

```
let num1 = Number(prompt("num1: "));
let num2 = prompt("num2: ");

let coc = num1/num2;

if (isNaN(coc)==true) {
  console.log("División no definida");
} else {
  console.log("Div es igual a: " + coc);
}

num1 = 4564.34567;
console.log(num1);
console.log(num1.toFixed(2)); // 4564.35
console.log(num1);
```

Ejemplos completos en los ficheros `ejemplo026.html` y `ejemplo026.js`.

3.3. Arrays

Puede consultar estos dos enlaces:

- [Uniwebsidad: Funciones y propiedades básicas de JavaScript](#)
- [W3schools: JavaScript Array Methods](#)

Métodos principales a conocer:

- `concat()` : concatena arrays
- `join(separador)` : hace lo contrario a `split` con las cadenas, devuelve un string con los elementos del array separados por `separador`.
- `pop()` : extrae un elemento al final del array y lo devuelve como resultado de la función.
- `push()` : inserta uno o varios elementos al final del array.
- `shift()` y `unshift()` : similares a `push` y `pop` pero extraen e insertan al principio del array.
- `reverse()` : modifica el array invirtiendo el orden de los elementos.
- `toReversed()` : para invertir los elementos **sin modificar el original**.
- `slice(ini, fin)` : Selecciona los elementos desde `ini` hasta `fin` (sin incluirlo) y devuelve ese array como resultado sin modificar el original.
- `splice(ini, nborrar, ele1, ele2, ...)` : permite insertar elementos nuevos (`ele1, ele2, ...`) en la posición `ini`. También permite eliminar `nborrar` elementos desde la posición `ini`. https://www.w3schools.com/js/js_array_methods.asp#mark_splice

Ejemplos completos en los ficheros `ejemplo030.html` y `ejemplo030.js`.

3.3.1. ORDENAR ARRAYS

El método `sort()` ordena alfabéticamente (de forma ascendente). Si desea ordenar de forma descendente puede usar el método `reverse()` después de la ordenación.

```
// The sort() method sorts an array alphabetically: LO MODIFICA
const fruits = ["Banana", "Orange", "Apple", "Mango"];
console.log(fruits);
```

```
fruits.sort();
console.log(fruits);
```

Como la **ordenación por defecto es alfanumérica** si desea ordenar números necesita una función auxiliar que indique el criterio de ordenación:

```
// ordenar números: hace falta función de comparación
// si negativo a menor, si positivo b menor, si cero no intercambio
function compara(a, b){
    return a - b;
}
let points = [40, 100, 1, 5, 25, 10];
console.log(points);
points.sort(compara);
console.log(points);
```

Al igual que `reverse()` y `toReversed()` existe `toSorted()`. Con este último el original se mantiene y se devuelve una copia ordenada.

Tiene ejemplos en los ficheros `ejemplo031.html` y `ejemplo031.js`.

3.3.2. BUSCAR EN ARRAYS

Algunos métodos de búsqueda los puede encontrar en este enlace: [W3schools: Array Find and Search Methods](#). Los básicos son:

- `indexOf(ele)`: busca `ele` en el array y devuelve la posición de su primera ocurrencia (o -1 si no lo encuentra).
- `lastIndexOf()`: similar al anterior pero buscando desde el final.
- `includes()`: comprueba si el elemento está incluido y devuelve `true` o `false`.

Tiene ejemplos en los ficheros `ejemplo032.html` y `ejemplo032.js`.

4.2.4 4. Funciones

4.1. Función sin parámetros

En primer lugar se presenta una función sin parámetros y que no retorna nada. Vea como se define la función:

```
// Definición función
function linea () {
    for (let i=0; i < 80; i++)
        document.write('*');
    document.write("<br/>");
}
```

Y ahora como se utiliza, su llamada o invocación:

```
// Invocación función
linea();
document.write("Hola Mundo! <br>");
linea();
```

4.2. Función con parámetros

Sería deseable en el ejemplo previo que se puedan generar líneas de distintas longitudes con esa función. Para ello la función debe recibir de alguna forma la longitud de la línea, y la forma de hacerlo es mediante un **parámetro** que se especifica en los paréntesis de la definición:

```
// Invocación función
function linea01 (tam) {
    for (let i=0; i < tam; i++)
        document.write('*');
    document.write("<br/>");
}
```

Observe cómo se añade el parámetro `tam` en los paréntesis, y cómo se usa como límite en el bucle `for` para indicar el número de repeticiones.

Si la función recibe un parámetro hay que modificar la llamada o invocación e indicar qué valor tomará ese parámetro. Vea un ejemplo usando un número o una variable:

```
linea01(5);
longitud = 40;
linea01(longitud);
```

Se podría mejorar la función con un segundo parámetro, el carácter que se usa para generar la línea:

```
// función con dos parámetros
function linea02 (tam, letra) {
  for (let i=0; i < tam; i++)
    document.write(letra);
  document.write("<br/>");
}
```

Y la llamada ahora se modifica para indicar ese segundo parámetro:

```
linea02(40, "=");
let longitud = prompt("Tamaño");
let car = prompt("carácter: ");
linea02(longitud, car);
```

También es posible invocar la función `linea02` desde otra función. En este ejemplo se invoca la función `linea02` en el bucle `for` de la función `piramide`

```
// función que usa otra función
function piramide(altura) {
  for (let i = 1; i <= altura; i++) {
    linea02(i, "*");
  }
}

let altura = prompt("Altura: ");
piramide(altura);
```

Tiene los ejemplos en los ficheros `ejemplo040.html` y `ejemplo040.js`.

4.3. Función que devuelve (retorna) resultado

Si una función realiza algún tipo de cálculo el valor calculado se devuelve como resultado de la función. Como ejemplo se propone una función que calcula la media de un array recibido como parámetro:

```
function media(lista) {
  let suma = 0;
  for (let i = 0; i < lista.length; i++) {
    suma += lista[i]
  }
  return suma/lista.length ;
}
```

Observe que se calcula la media y ese valor calculado se indica en la sentencia `return` que cierra la función. Para usar el valor devuelto por la función es necesario asignar la invocación de la función a una variable (o usarla en una expresión).

Vea como el resultado de la llamada `media(edades_A)` se asigna a la variable `medA`

```
let edades_A = [23, 34, 45, 19, 25, 31];
// se asigna el valor de retorno de la función a medA
let medA = media(edades_A);
```

Un ejemplo del uso de esta función con dos arrays distintos:

```
let edades_A = [23, 34, 45, 19, 25, 31];
let medA = media(edades_A);

let edades_B = [45, 51, 37, 28, 18];
let medB = media(edades_B);

if (medA>medB)
  console.log("A > B");
else if (medA<medB)
  console.log("A < B");
else
  console.log("A = B");
```

Tiene ejemplos en los ficheros `ejemplo041.html` y `ejemplo041.js`.

Para ver la importancia de devolver un valor como resultado de la función intente hacer este ejemplo previo sin devolver el valor.

4.2.5 5. Objetos

Haremos una breve introducción al tema de los objetos en JavaScript. Tiene una referencia adicional en [W3Schools: JavaScript Objects](#)

Los objetos son variables que contienen **propiedades** y **métodos**. Los objetos se delimitan con llaves. Veamos un primer ejemplo de objeto con propiedades

```
let alumno = {
  nombre: "Ana",
  apellidos: "Milton",
  id: 5566
};
```

Las propiedades del objeto son pares `key:value` (nombre: valor), si hay varias separadas por comas. Se puede acceder a las propiedades del objeto de dos formas, notación de punto y de corchetes:

```
console.log(alumno.nombre); // 1)
console.log(alumno["nombre"]); // 2)
```

5.1. Recorrer propiedades de un objeto

Para recorrer las propiedades de un objeto se puede usar una variante del bucle `for (prop in obj)`. Este bucle permite acceder al nombre de la propiedad y a su valor:

```
for (let propiedad in objeto) {
  console.log(propiedad, objeto[propiedad]);
}
```

Ejemplo:

```
for (let prop in alumno) {
  console.log(prop + ":", alumno[prop]);
}
```

Esto código genera en consola la salida:

```
nombre: Ana
apellidos: Milton
id: 5566
```

5.2. Arrays de objetos

Puede definir un array donde cada elemento sea un objeto:

```
let alumnos = [
  {nombre: "Ana Tendero", edad: 27 },
  {nombre: "Luis Fornes", edad: 16 },
  {nombre: "Bet Villa", edad: 21 }
];
```

Al ser un array lo puede recorrer con un bucle `for` y acceder en cada iteración a las propiedades de cada objeto. Vea un ejemplo del cálculo de la media:

```
function media(alumnos) {
  var suma = 0;
  for (var i = 0; i < alumnos.length; i++) {
    suma += alumnos[i].edad;
  }
  return suma/alumnos.length;
}
document.write("<br>Media: ", media(alumnos) );
```

5.3. Objetos y Métodos

Además de propiedades, los objetos tienen métodos. Los métodos son acciones a realizar sobre los objetos. Ya ha usado objetos y métodos previamente:

- Al usar variables de tipo string y sus métodos predefinidos

```
let nombre = "Alfredo";
console.log( nombre.length ); // propiedad
console.log( nombre.indexOf("l") ); // método
```

- O al usar métodos de arrays

```
let temperaturas = [34, 37, 31, 28];
console.log( temperaturas.length ); // propiedad
console.log( temperaturas.sort() ); // método
```

Vea un ejemplo cómo añadir un método a un objeto. Observe el uso de la palabra reservada `this` en la función: se refiere al objeto dónde se está invocando ese método y permite acceder a las propiedades del mismo, `this.firstName` y `this.lastName`:

```
let person = { // definición
  firstName: "John",
  lastName: "Doe",
  id: 5566,
  fullName: function() { // uso de this
    return this.firstName + " " +
      this.lastName;
  }
};
```

Para invocar el método del objeto `person` se usa el nombre del método con los paréntesis (como en funciones):

```
// invocación
console.log( person.fullName() );
```

Otro ejemplo: se define un objeto vacío al que se le van añadiendo propiedades (`id`, `modulos`, `notas`) y un método (`media`):

```
let estudiante = {}; // se define objeto
estudiante.id = "ASIR-1-001";
// algunas propiedades son arrays
estudiante.modulos = ["LMSGI", "PAR", "FOL", "ISO"];
estudiante.notas = [4, 6, 9, 2];
// se añade método
estudiante.media = function () {
  let suma = 0;
  for (let i = 0; i < this.notas.length; i++)
    suma += this.notas[i];
  return suma/this.notas.length;
}

// accedemos a las propiedades
console.log(estudiante.id);
for (let i = 0; i < estudiante.modulos.length; i++)
  console.log(estudiante.modulos[i], ":", estudiante.notas[i]);

//invocación del método
console.log("Media:", estudiante.media());
```

Tiene estos ejemplos en los ficheros `ejemplo050.html` y `ejemplo050.js`.

De todas formas no se pretende con esta introducción mostrar teoría de clases, objetos, herencia, etc. En ECMAScript el soporte de clases está en plena evolución. Basta por ahora saber usar las propiedades y métodos que nos proporciona el lenguaje y su notación.

4.2.6 6. Anexos

6.1. Math y random

El objeto `Math` incluye varios métodos matemáticos. Ejemplo de uso de alguno de ellos:

```
console.log(Math.sqrt(14));
let media = 5.6;
let m = Math.floor(media);
console.log(m);
m = Math.round(media);
console.log(m);
```

Otro es `Math.random()` que devuelve un número aleatorio entre 0 y 1 (sin incluir el 1).

Si desea un nº aleatorio entre 1 y 10 vea un primer ejemplo:

```
// random devuelve entre 0 y 1 con decimales...
// y necesitaremos floor para redondear a nº entero,
let aleat = Math.floor( (Math.random()*10) + 1 );
console.log(aleat)
```

Si desea que el número aleatorio se genere entre un `min` y un `max`:

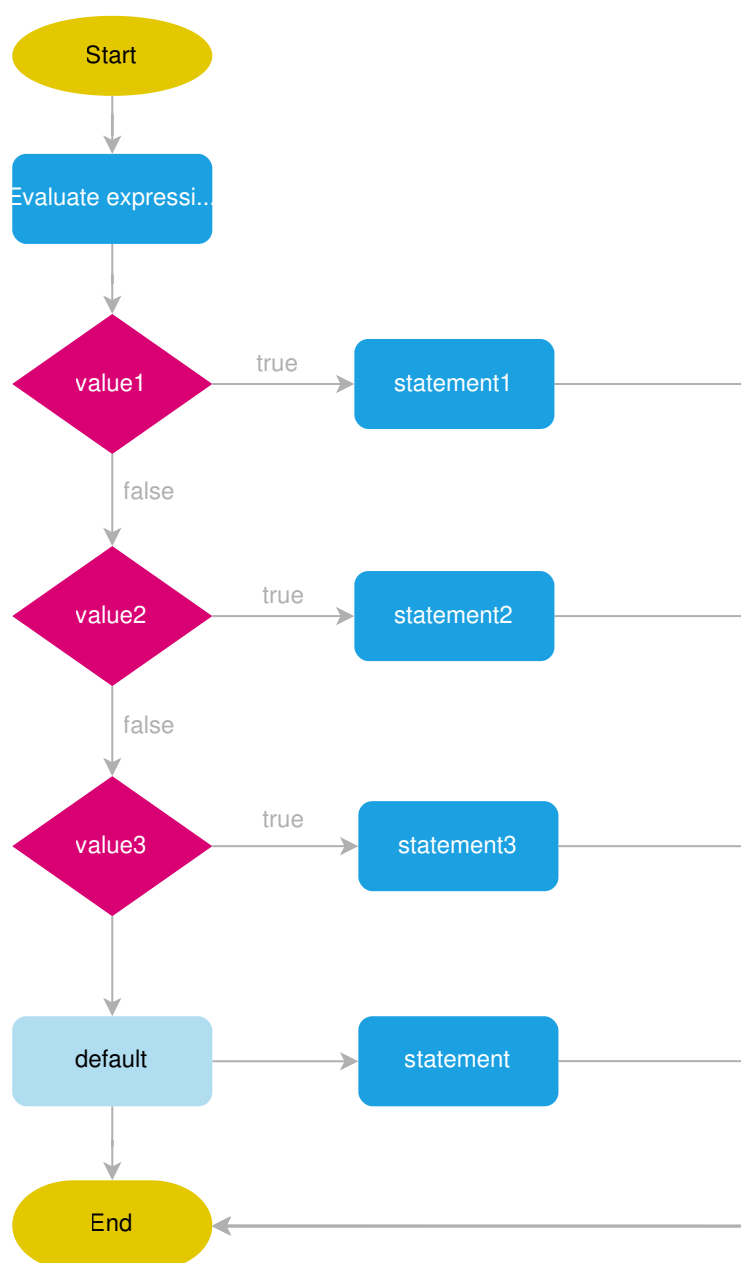
```
/* aleatorio entre variables min y max */
let max = 100;
let min = 1;
aleat = Math.floor(Math.random()*(max - min + 1)) + min;
```

Incluso se puede crear una función para encapsular dicho cálculo y que reciba como parámetros el máximo y el mínimo, y que devuelva como resultado el número aleatorio:

```
/* o creamos una función */
function getRndInteger(min, max) {
  return Math.floor(Math.random()*(max - min + 1)) + min;
}
aleat = getRndInteger(1,100);
```

Tiene estos ejemplos en los ficheros `ejemplo051.html` y `ejemplo051.js`.

6.2. Selectiva múltiple



Puede consultar el uso de esta estructura en este enlace: <https://www.javascripttutorial.net/javascript-switch-case/>. Se usa a veces como simplificación de una estructura `if-elseif`

Con `switch case`

```

switch ( numero ) {
  case 5:
    ...
    break;
  case 8:
    ...
    break;
  case 20:
    ...
    break;
  default:
    ...
    break;
}

```

Y con `if else if`

```

if ( numero == 5 ) {
    ...
}
else if(numero == 8) {
    ...
}
else if(numero == 20) {
    ...
}
else {
    ...
}

```

6.3. Bucles while y do while

- while: <https://www.javascripttutorial.net/javascript-while-loop/>
- do while: <https://www.javascripttutorial.net/javascript-do-while/>

La diferencia principal es cuándo se evalúa la condición: antes o después de la primera ejecución del bucle.

```

do {
    // statements;
} while(expression);

```

```

while (expression) {
    // statements
}

```

6.4. Variable contador, acumulador, bandera

- Contador: variable cuyo valor se incrementa o decrementa en una cantidad constante cada vez que se produce un determinado suceso o acción
- Acumulador: variable que suma sobre sí misma un conjunto de valores, para de esta manera tener la suma de todos ellos en una sola variable.
- Tenga en cuenta que debe iniciar contador y acumulador
- antes del bucle
- acumuladores a cero si suma, a uno si producto.
- banderas (flag, interruptor o conmutador): variable que puede tomar dos valores (verdadero o falso) a lo largo de la ejecución del programa y permite comunicar información de una parte a otra del mismo para bifurcar el código.

Ejemplos:

```

// contador: ejemplo, variable que cuenta el número de aprobados
var nAprobados = 0;
for (i = 0; i < notas.length; i++) {
    if (notas[i] >= 5)
        nAprobados++;
}
// acumulador: ejemplo, variable que acumula la suma
suma = 0;
for (i = 0; i < notas.length; i++) {
    suma += notas[i];
}
// bandera o flag: ejemplo, detecta si hay algún suspenso
haySuspendos = false;
for (i = 0; i < notas.length; i++) {
    if (notas[i] < 0)
        haySuspendos = true;
}

```

6.5. Estilos de llaves y tabulación

- Conviene elegir un estilo, y usar siempre el mismo.
- Sobre el sangrado, hay dos corrientes:
- usar el carácter tabulador
- Usar espacios (2, 4...)

Whitesmith <pre>if(condicion) { sentencia0; sentencia1; }</pre>	GNU <pre>if(condicion) { sentencia0; sentencia1; }</pre>
BSD/Allman <pre>if(condicion) { sentencia0; sentencia1; }</pre>	Kernighan&Ritchie/Kernel <pre>if (condicion) { sentencia0; sentencia1; }</pre>

6.6. Métodos de arrays avanzados

Vamos a presentar los métodos:

- `forEach`
- `every` y `some`
- `map` y `filter`

Estos métodos reciben como parámetro una función llamada de retorno. Y esa función se aplicará a cada uno de los elementos del array. La función de retorno o comprobación puede tener hasta tres parámetros:

1. El primero es el elemento: elemento del array procesado
2. El segundo es el índice: índice del elemento procesado
3. El tercero es el array: el array (completo) que se está procesando.

```
function fRetorno(elemento, indice, array) {
    ...
}
```

6.6.1. FOREACH

Vea como funciona `forEach` en los ejemplos de los ficheros `ejemplo060.html` y `ejemplo060.js`. En ellos se aprecia cómo se construye una función de retorno y como se invoca el método `forEach`.

El método `forEach` va llamando a la función de retorno para cada elemento del array:

```
let numeros = [4, 9, 16, 25];

// uno, por ser el 1º es el elemento
function fcallback1(ele) {
    document.write("<br>llamada a callback1: ", ele);
}
// LLAMADA A forEach,
// el nombre de la función es el parámetro
numeros.forEach( fcallback1 );
```

Se puede usar el método `para` recorrer un array de objetos y mostrar sus propiedades:

```
let alumnos = [
    {nombre: "Ana Tendero", edad: 27 },
    {nombre: "Luis Fornes", edad: 19 },
    {nombre: "Bet Villa", edad: 21 }
];

function muestraClase(val, index) {
    var let = "<br> pos " + index + " > " +
```

```

    val.nombre + " " + val.edad;
    document.write(let);
  }

  alumnos.forEach(muestraClase);

```

6.6.2. EVERY Y SOME

- `every`: devuelve `true` si la función de comprobación que recibe `every` devuelve `true` **para todos** los valores del array.
- `some`: devuelve `true` si la función de comprobación que recibe `some` devuelve `true` **para alguno** de los valores del array.

Es importante destacar que la función de comprobación debe devolver `true` o `false` cuando procesa un elemento.

```

// función de comprobación
function compruebaAdulto(edad) {
  return edad >= 18; // DEVUELVE true o false
}

let edades = [32, 33, 16, 40];
document.write("<br>Edades: ", edades);
let todos_adultos = edades.every(compruebaAdulto);
document.write("<br>Todos adultos: ", todos_adultos);
let algun_adulto = edades.some(compruebaAdulto);
document.write("<br>Algún Adulto: ", algun_adulto);

```

6.6.3. MAP Y FILTER

El método `map()` crea un nuevo array con los resultados de la llamada a la función de retorno aplicados a cada uno de sus elementos.

En este primer ejemplo se usa la función `sqrt` de `Math`:

```

let valores= [1, 4, 9];

let raices = valores.map(Math.sqrt);
console.log("Raices: ", raices)

```

Y en este otro ejemplo se usa una función definida por el usuario. Observe que el parámetro de `map` es el nombre de la función (sin paréntesis)

```

function duplica(a) {
  return a*2;
}

let dobles = valores.map(duplica);
console.log("dobles: ", dobles)

```

El método `filter()` crea un nuevo array con los elementos que pasen el test de la función de retorno (dicha función debe devolver `true|false`)

```

edades = [32, 33, 16, 40];
console.log("edades: ", edades);

// función de retorno
function esAdulto(edad) {
  return (edad >= 18); // debe devolver true|false
}

let adultos = edades.filter(esAdulto);
console.log("adultos: ", adultos)

// función flecha, nueva sintaxis en ES6
let menores = edades.filter( a => a<18 );
console.log("menores: ", menores)

```

Tiene el código de los ejemplos en los ficheros `ejemplo060.html` y `ejemplo060.js`.

4.3 Introducción JavaScript 03 - DOM

4.3.1 1. Introducción DOM

Puede obtener una aproximación en este enlace [W3Schools: JavaScript HTML DOM](#)

El modelo DOM es una herramienta esencial para la manipulación y dinamización de páginas web. A través de JavaScript y la API del DOM, los desarrolladores pueden crear experiencias interactivas sin necesidad de recargar la página, optimizando la usabilidad y la experiencia del usuario.

El Modelo de Objetos del Documento (DOM) es una interfaz de programación que representa la estructura de un documento HTML o XML como un árbol jerárquico de nodos. Cada elemento del documento, como etiquetas, atributos, texto y comentarios, se convierte en un nodo dentro de este árbol, permitiendo que los lenguajes de programación, como JavaScript, accedan y manipulen dinámicamente el contenido y la estructura del documento.

Cuando un navegador carga una página web, analiza el código HTML y lo convierte en una estructura de nodos organizada en un árbol. Cada etiqueta HTML se representa como un nodo, que puede contener otros nodos como elementos hijos o atributos. JavaScript puede acceder y modificar estos nodos a través de métodos y propiedades del API del DOM.

4.3.2 2. Accesos "directos"

DOM define algunas propiedades que nos permiten acceder directamente a información y elementos de la página HTML. Entre ellos por ejemplo:

- `document.URL`: devuelve la URL del documento.
- `document.title`: devuelve el valor del elemento `<title>`.
- `document.links`: devuelve una colección con todos los elementos con atributo `href`.
- `document.images`: devuelve una colección con todos los elementos `<img>` del documento.
- `document.forms`: devuelve una colección con todos los elementos `<form>` del documento.
- `document.lastModified`: devuelve la hora de modificación del documento.

Tiene un ejemplo de su uso en el fichero `dom-001.html` y `dom-001.js`.

4.3.3 3. Acceder a elementos HTML

3.1. Acceso con métodos

El primer paso para poder añadir elementos, modificarlos, etc. es identificar los elementos a procesar. Para ello se definen métodos para poder acceder a los elementos HTML que cumplan alguna condición: por id, por clase, por etiqueta, etc. Estos métodos devuelven un elemento o una colección de elementos (*HTMLCollection*, similar a un array).

Alguno de estos métodos:

- `getElementById(id)` => retorna **un elemento** con el id indicado
- `getElementsByName(tag)` => retorna **todos los elementos** con esa etiqueta, una colección
- `getElementsByClassName(class)` => retorna **todos los elementos** de esa clase, una colección.

3.2. Acceso al contenido o estilo

Y una vez identificado el elemento a procesar podemos acceder a su contenido o estilo con las propiedades:

- `.textContent`: permite acceder al contenido de un elemento HTML (sin etiquetas interiores)
- `.innerHTML`: permite acceder al contenido de un elemento HTML (incluidas las etiquetas interiores).
- `.style`: permite acceder a los estilos CSS de un elemento HTML y modificarlos.

Puede observar el uso de estos métodos y propiedades en los ficheros `dom-002.html` y `dom-002.js`. En el fichero HTML se añade un botón al que se le asocia la ejecución del código contenido en la función `test()`. El resultado se mostrará en consola.

En ese caso observe el uso de un método `.getElementsByTagName()` que devuelve una colección de elementos y el uso de `textContent` e `innerHTML`:

```
// ejemplo acceso por etiqueta, devuelve colección elementos
var parrafos = document.getElementsByTagName("p");

// mostramos párrafos en consola
// .innerHTML => ver como los párrafos llevan el HTML "interno"
for (let i = 0; i < parrafos.length; i++) {
  console.log(parrafos[i].innerHTML);
  console.log(parrafos[i].textContent);
}
```

Y el uso de `.style` y `.getElementById()` (que devuelve un único valor):

```
// acceso por ID: devuelve un único elemento
let contenido = document.getElementById("content");
contenido.style.border = "4px blue double";
```

3.3. Métodos con selector CSS

Hay métodos para identificar elementos HTML que usan como **parámetro un selector CSS** lo cual proporciona una gran flexibilidad si los sabe utilizar.

- `querySelector(selectorCSS)` => retorna **el primer elemento** que cumpla el selector CSS.
- `querySelectorAll(selectorCSS)` => retorna **todos los elementos** que cumplan el selector CSS.

Vea ejemplos del uso de estos métodos en los ficheros `dom-003.html` y `dom-003.js`.

```
let parrafos = document.querySelectorAll("section p:nth-of-type(odd)");
for (let i = 0; i < parrafos.length; i++) {
  parrafos[i].style.color = "red";
}
```

4.3.4 4. Acceder a los atributos HTML

Los atributos de un elemento HTML son convertidos por el navegador en propiedades de un objeto. Para acceder a ellos puede usar los métodos:

- `.getAttribute(atributo)` : devuelve como resultado el valor del `atributo`
- `.setAttribute(atributo, nuevoValor)` : permite asignarle el valor `nuevoValor` a `atributo`.

```
let h1 = document.querySelector("h1");
h1.setAttribute("title", "título generado desde script");

let titulos = document.querySelectorAll("[title]");
for (let i = 0; i < titulos.length; i++) {
  texto = titulos[i].getAttribute("title");
  console.log(texto);
}
```

4.1. Casos especiales

Al **acceder directamente a propiedades del DOM** sin usar los métodos anteriores el valor retornado no siempre coincide con el valor del atributo HTML.

Es el caso del atributo `href` de un enlace:

- Si accede a su valor con `.getAttribute(href)` se obtiene el valor del atributo HTML (puede ser un enlace relativo)
- Si accede con la propiedad DOM `.href` se obtiene siempre la URL absoluta de ese enlace.

Vea la diferencia en este ejemplo:

```
let enlaces = document.querySelectorAll("nav a");
for (let i = 0; i < enlaces.length; i++) {
  console.log("con href: ", enlaces[i].href);
  console.log("con getAttribute: ", enlaces[i].getAttribute("href"));
}
```

Otro caso particular es el atributo `class`. Se puede acceder a su valor

- usando los métodos `.setAttribute()` y `.getAttribute()`
- usando directamente la propiedad `.className` (no se puede usar `.class` como nombre de la propiedad porque es una palabra reservada)

```
ele.className = "xxx";
ele.className += " xxx";
```

En los dos casos añadir una clase a un elemento HTML (o eliminarla o comprobarla) no es inmediato ya que el atributo `class` puede contener varios valores (`class="revisar importante actual"`). En estas situaciones se usa `.classList` y sus métodos:

- `.classList.add(clase)`
- `.classList.remove(clase)`
- `.classList.contains(clase)`

Ejemplo de su uso:

```
let h2 = document.querySelector("section h2");
console.log(h2.className);
h2.classList.add("nueva");
console.log(h2.className);
console.log(h2.classList.contains("nueva"));
```

Tiene todos los ejemplos de uso de atributos en los ficheros `dom-004.html` y `dom-004.js`.

4.3.5 5. Estilos

Ya se ha comentado que para acceder al valor de una propiedad CSS de un elemento HTML se usa la propiedad `style`:

```
element.style.propiedad = "valordelapropiedad";
console.log(element.style.propiedad);
```

Pero si el nombre de la propiedad CSS tiene un guión se transforma el nombre de la propiedad: se eliminan los guiones y se capitaliza la propiedad usando la notación **camelCase**. Por ejemplo:

- `font-size => fontSize`.
- `background-color => backgroundColor`
- `list-style-type => listStyleType`

Ejemplo de uso:

```
let h1 = document.querySelector("h1");
h1.style.backgroundColor = "yellow";
h1.style.textDecoration = "underline red 4px double";
```

Otra forma "indirecta" de cambiar el estilo es usando clases HTML:

1. Se define una regla CSS en la hoja de estilos asociada a una clase.
2. Se modifica el atributo `class` del elemento para asignarle dicha clase.

Ejemplo de uso:

```
// usando clases que tengan estilos asociados
let h2 = document.querySelector("section h2");
h2.classList.add("destacar");
```

Tiene el código de estos ejemplos en los ficheros `dom-005.html` y `dom-005.js`.

4.3.6 6. Trabajar con elementos HTML

6.1. Añadir un elemento

Para añadir un nuevo elemento HTML podemos pensar en tres fases:

1. Primero hay que crearlo con `.createElement()` especificando el tipo del elemento

```
let nuevo = document.createElement("p");
```

2. Posteriormente se le proporciona contenido (y estilos si hiciera falta)

```
nuevo.textContent = "texto que se añade al elemento";
nuevo.style.color = "blue";
```

3. Y por último falta decidir dónde insertarlo. Opciones:

4. Antes o después de un cierto elemento (que llamaremos "intermedio" o referencia). Para ello usará `.after(nuevo)` o `.before(nuevo)` :

```
intermedio.after(nuevo);
intermedio.before(nuevo);
```

5. El primero o el último en un elemento contenedor (que llamaremos "padre"). En este caso usará `.append(nuevo)` (último) o `.prepend(nuevo)` (primero)

```
padre.prepend(nuevo); // añade nuevo como primer hijo
padre.append(nuevo)  // añade nuevo como último hijo
```

6.2. Borrar un elemento

Para borrar un elemento basta obtener su referencia y aplicarle el método `.remove()` sin parámetros:

```
let h2 = document.querySelector("nav h2");
h2.remove();
```

Tiene el código de estos ejemplos en los ficheros `dom-006.html` y `dom-006.js`.

6.3. Mover un elemento

Mover es similar al tercer paso de crear un nuevo elemento: sólo necesita la referencia al elemento a mover y dónde quiere moverlo.

4.3.7 7. Eventos

Un evento es cualquier acción o suceso que ocurre en una página web y que puede ser detectado y manejado con código JavaScript. Los eventos pueden ser generados por el usuario (clics, teclas presionadas, movimientos del ratón), por el sistema (carga de la página, errores) o por interacciones automáticas.

7.1. Tipos de eventos

Los eventos se pueden clasificar en varias categorías según su naturaleza:

- Eventos del mouse:
 - click: Se activa cuando el usuario hace clic en un elemento.
 - dblclick: Se dispara cuando se hace doble clic en un elemento.
 - ...
- Eventos del teclado:
 - keydown: Se activa cuando el usuario presiona una tecla.
 - keyup: Se dispara cuando el usuario suelta una tecla.
- Eventos de formulario:
 - submit: Se activa cuando se envía un formulario.
 - change: Se dispara cuando el valor de un campo de entrada cambia.
 - focus: Se activa cuando un campo recibe el foco.
 - ...
- Eventos de la ventana y del documento:
 - load: Se dispara cuando la página o un recurso se ha cargado completamente.
 - resize: Se activa cuando se redimensiona la ventana del navegador.
 - ...

Tiene una lista de posibles eventos en este enlace de [W3Schools: HTML DOM Events](https://www.w3schools.com/js/default_events.asp).

7.2. Asociar eventos a código javascript

7.2.1. FORMA 01

La forma más sencilla y directa (pero no por ello recomendable) de asociar código a un evento es incluir en el elemento HTML el evento como atributo y el código a ejecutar como valor del atributo:

```
<elemento evento="códigoJavaScript">
```

Este podría ser un ejemplo inicial donde al hacer clic sobre el botón se sustituye el texto del párrafo con `id="demo"` por la fecha actual:

```
<p id="demo">¿Hora?</p>
<button onclick="getElementById('demo').innerHTML=Date()">Ver hora 01</button>
```

Es recomendable crear una función con el código javascript e invocarla en el atributo del evento:

```
function verHora() {
  let demo = document.getElementById("demo");
  demo.innerHTML=Date();
  demo.style.color="blue";
}
```

Y el HTML queda

```
<button onclick="verHora()">Ver hora 02</button>
```

7.2.2. FORMA 02

Otra forma de asociar eventos a código es identificar el botón (por ejemplo con un `id`) en el código HTML, y realizar en el código JavaScript la asociación evento-función. De esa forma en el código HTML no se indica el evento ni hay código JavaScript.

El código HTML queda así:

```
<button id="ver03">Ver hora 03</button>
```

Y en el código JavaScript se asocia al evento `onclick` la función `verHora` (observe que se asigna la función sin paréntesis, no se está invocando):

```
// asocia onclick a verHora
btn = document.getElementById('ver03')
btn.onclick=verHora;
```

7.2.3. FORMA 03: `addEventListener`

Una última posibilidad es utilizar el método `.addEventListener()` que es la forma que tiene más posibilidades (aunque aquí se mostrará su uso básico).

En esta forma primero se localiza el elemento HTML al que se le asocia el evento, y a la hora de asociar el evento usamos el método

`.addEventListener()` que recibe dos parámetros:

1. El **evento** que se asocia ((observe que en el ejemplo el parámetro es `click` y no `onclick`).
2. La **función** que se asocia al evento.

Lo puede observar en este ejemplo:

```
var btn = document.getElementById('ver04');
btn.addEventListener("click", verHora);
```

Observe que el evento no lleva el prefijo `on`. Tiene el código de estos ejemplos en los ficheros `dom-010.html` y `dom-010.js`.

Y más información sobre este método en este enlace [W3Schools: HTML DOM Element addEventListener\(\)](#)

7.3. Ejemplo uso eventos y formularios

Para validar los campos de un formulario se puede usar el evento `onSubmit`. Al asociar este evento a una función esta se ejecuta al pulsar el botón de envío del formulario. Si la función devuelve `false` el formulario no se envía.

Vea un ejemplo de formulario de alta:

- el campo clave se solicita dos veces (inputs `pwd1` y `pwd2`)
- se añade el atributo `onsubmit` en el formulario que retorna el resultado de la función `checkForm()`

```
<form id="form" onsubmit="return checkForm();" action="val.php">
  <label>Usuario: <input id="user" type="text" name="user" /></label>
  <label>Clave: <input id="pwd1" type="password" name="pwd1"/></label>
  <label>Confirma:<input id="pwd2" type="password" name="pwd2"/></label>
  <input type="submit" value="Enviar" />
</form>
```

Y el código de la función de control `checkForm` que devuelve `false` si las claves son distintas.

```
function checkForm() {
  var pwd1 = document.getElementById("pwd1");
  var pwd2 = document.getElementById("pwd2");
  if (pwd1.value != pwd2.value) {
    alert("Error: clave y confirmación deben ser iguales");
    return false; //si onsubmit devuelve false no se envía formulario
  }
  return true;
}
```

Observe que se accede al valor de los `inputs` del formulario `pwd1` y `pwd2` con la propiedad `value`.

7.4. Eventos y la palabra reservada `this`

Es frecuente que quiera asociar **el mismo evento y función** a un conjunto de elementos. Y que la función se aplique sobre el elemento que genera el evento en cada momento.

En este caso la palabra reservada `this` permite acceder (desde la función asociada al evento) al elemento HTML que ha provocado el evento (y por tanto la ejecución de la función).

Si por ejemplo quiere asociar la función a todos los párrafos de la página HTML debe recorrerlos con un bucle asignando a cada párrafo el evento y una función:


```
var parrafos = document.querySelectorAll('p');
for (let i = 0; i < parrafos.length; i++) {
  parrafos[i].onclick = colorVerde;
  // en cada párrafo se asocia evento onclick a función colorVerde
}
```

Y vea la función `colorVerde` que cambia el color del elemento que la invoque. Como no está determinado que elemento HTML la invocará se usa la palabra reservada `this` en la expresión `this.style.color = "green";`:

```
function colorVerde() {
  this.style.color = "green";
}
```

Ahora cada vez que haga click sobre un párrafo cambiará el color de dicho párrafo.

4.3.8 8. Anexos

8.1. Generar un elemento HTML con hijos

El objetivo es crear un elemento HTML que dentro tendrá a su vez elementos HTML hijos. Puede ser por ejemplo una lista y sus elementos, o una tabla con filas, etc.

Tiene varias formas de abordar esta situación. Una de ellas sería:

1. Primero se crea el elemento HTML `contenedor`
2. Luego se añade dentro de `contenedor` los hijos (podría ser un bucle)
3. se crea el elemento HTML `hijo`
4. se le da contenido, estilo, etc.
5. se inserta `hijo` en `contenedor`
6. Finalmente se inserta `contenedor` en la página HTML

```
function creaLista01() {
  var lista = document.createElement("ul"); // 1

  for (let i = 0; i < 5; i++) {
    item = document.createElement("li"); // 2.1
    item.textContent = "item " + i; // 2.2
    lista.append(item); // 2.3
  }
  var destino = document.getElementById("resultado");
  destino.append(lista); // 3
}
```

Otra forma de conseguir el mismo objetivo sería:

1. Primero se crea el elemento HTML `contenedor`
2. Se genera una cadena o string con todo el contenido de `contenedor` (incluyendo las etiquetas HTML de los hijos)
3. Se añade esa cadena al contenido del `contenedor` usando `innerHTML` ya que se incluyen en la cadena etiquetas HTML).
4. Finalmente se inserta `contenedor` en la página HTML.

```
function creaLista02() {
  var destino = document.getElementById("resultado");
  var lista = document.createElement("ul");

  for (let i = 0; i < 5; i++) {
    lista.innerHTML += "<li> item " + i + "</li>";
  }
  lista.innerHTML += "</ul>";
  destino.append(lista);
}
```

8.2. Eventos, APIs y JSON

Vea un ejemplo de consulta a una API desde JavaScript. En este caso se asocia al evento `onclick` la función `generaUsuario()`.

Esta función hace la consulta a la API usando en la URL como parámetro el valor del input del formulario con `id="usuario"`. Para ello se usa el método `.fetch`.

La respuesta de la API se convierte a formato JSON y se utiliza para construir una cadena `cad`. Esa cadena contiene el código HTML de dos filas de una tabla, una con los encabezados y otra con la respuesta de la API. Esta cadena se inserta dentro de una tabla con id `tabla1` ya existente en la página HTML:

```
btn1 = document.querySelector("#busca");
btn1.onclick = generaUsuario;

function generaUsuario() {
  usuario = document.querySelector("#usuario");
  fetch("https://jsonplaceholder.typicode.com/users/" + usuario.value)
    .then((response) => response.json())
    .then((response) => {
      let cad = "<tr><th>Nombre</th><th>Email</th><th>Teléfono</th></tr>";
      cad += `<tr><td>${response.name}</td>
              <td>${response.email}</td>
              <td>${response.phone}</td></tr>`;

      document.getElementById("tabla1").innerHTML = cad;
    });
}
```

Este ejemplo es similar, pero la consulta a la API devuelve una lista de usuarios. Por ello se usa un bucle que va generando tantas filas de la tabla como usuarios:

```
btn2 = document.querySelector("#lista");
btn2.onclick = generaTabla;

function generaTabla() {
  console.log("genera Tabla")
  fetch("https://jsonplaceholder.typicode.com/users/")
    .then((response) => response.json())
    .then((response) => {
      let cad = "<tr><th>Nombre</th><th>Email</th><th>Teléfono</th></tr>";
      for (let usuario of response) {
        cad += `<tr><td>${usuario.name}</td>
                <td>${usuario.email}</td>
                <td>${usuario.phone}</td></tr>`;
      }
      document.getElementById("tabla1").innerHTML = cad;
    });
}
```

Tiene el código en los ficheros `index.html` y `app.js` de la carpeta `json_api`. Tiene más detalles del uso de esta API en este enlace: [JSON Placeholder](#)

8.3. Curiosidad: HTMLCollection vs NodeList

Los métodos `getElementsByTagName`, `getElementsByClassName` devuelven una colección HTML (*HTMLCollection*), pero el método `querySelectorAll` devuelve una lista de nodos (*NodeList*). De ellos nos interesa que

- Los objetos `NodeList` son **estáticos**.
- Los objetos `HTMLCollection` son **dinámicos** o vivos: se actualizan automáticamente si se modifica el documento.

Hay que tener cuidado al insertar o borrar elementos de un `HTMLCollection` usando un bucle. Vea en la siguiente captura lo que ocurre al borrar un nodo de una lista: cómo se actualiza la colección HTML y cómo no lo hace la lista de nodos:

```

>> var coleccionHTML = document.getElementsByTagName('p')
< undefined

>> var listaNodos = document.querySelectorAll('p')
< undefined

>> coleccionHTML
< ▶ HTMLCollection(4) [ p , p.revisar , p , p ]

>> listaNodos
< ▶ NodeList(4) [ p , p.revisar , p , p ]

>> listaNodos[0].remove()
< undefined

>> coleccionHTML
< ▶ HTMLCollection(3) [ p.revisar , p , p ]

>> listaNodos
< ▶ NodeList(4) [ p , p.revisar , p , p ]

```

Vea ejemplos de funciones que borran en bucle. En el primer caso se recorre siempre borrado el primero para no tener problemas:

```

function borraColeccion() { // HTML Collection
  var lista = document.getElementsByTagName("p");
  for (var i = lista.length - 1; i >= 0; i--) {
    lista[i].remove();
  }
}

function borraLista() { // con NodeList
  var lista = document.querySelectorAll("p");
  for (var i = 0; i < lista.length; i++) {
    lista[i].remove();
  }
}

```

4.3.9 9. Posibles ejercicios

- Mostrar cuántas cabeceras de nivel 2 hay en el artículo. De ellas, cuántas tienen una longitud menor de 8, entre 8 y 50 y mayor de 50 caracteres.
- Estilos
 - Modificar el color del primer `<li>` de cada lista a rojo.
 - Poner un borde inferior a todos las cabeceras `h2` y `h3` de la página.
- Modificar la clase de los `h2` para que tengan la clase `indice`.
- Modificar todos los elementos de la clase `resumen` y añadirle la clase `destacar`.
- Añadir elementos en una lista
 - Al principio de una lista.
 - Al final de una lista.
 - Antes de un elemento con `id=intermedio`.
- Si la página no tiene `footer` añadirlo con un `h2`.
- Borrar los elementos de la página que al pulsar un botón añade al final de la página una lista con todas las palabras importantes (`strong`).
- Código que al pulsar un botón crea una lista con las abreviaturas existentes y su significado. Si se vuelve a pulsar el botón y la lista existe la borra.

4.4 JavaScript Abreviado

4.4.1 1. Introducción

Este documento sólo pretende dar una introducción rápida a JavaScript.

- En la carpeta `00_iniciales` tiene ejemplos básicos de inclusión de código javascript en una página HTML.
- En la carpeta `01_formulario` tiene ejemplos de validación de formulario con JavaScript.
- En la carpeta `02_DOM` tiene ejemplos del uso del modelo DOM para acceder a los elementos HTML, su contenidos, sus atributos, estilos ,etc.
- En la carpeta `03_eventos` tiene ejemplos del uso de eventos.
- En la carpeta `00_hora` tiene un ejemplo del uso de Javascript para modificar estilos o contenidos en función de la hora del día.

4.4.2 2. Ejemplos básicos

2.1. Modificar contenido

ejemplo01.html

```
<h2>What Can JavaScript Do?</h2>

<p id="demo">JavaScript <strong>can change</strong> HTML content.</p>

<button type="button"
  onclick='document.getElementById("demo").innerHTML = "Hello JS!"'>
  Click Me!
</button>
```

2.2. Modificar atributo

ejemplo02.html

```
<button
  onclick="document.getElementById('myImage').src='figuras/pic_bulbon.gif'">
  Turn on the light
</button>



<button
  onclick="document.getElementById('myImage').src='figuras/pic_bulboff.gif'">
  Turn off the light
</button>
```

2.3. Modificar estilo CSS

ejemplo04.html

```
<button
  type="button"
  onclick="document.getElementById('demo').style.fontSize='35px'">
  Click Me!
</button>
```

4.4.3 3. Referencias

- JavaScript Tutorial W3Schools: <https://www.w3schools.com/js/default.asp>
- Introducción a JavaScript en Uniwebsidad ahora en archive.org
- Eloquent JavaScript: <https://eloquentjs-es.thedojo.mx/>
- Manual JavaScript : <https://desarrolloweb.com/home/javascript>

4.4.4 4. Entorno de trabajo

En estos ejemplos se ha usado una máquina virtual con Linux y VSCode como editor.

4.5 Linter JavaScript: eslint 24/25

4.5.1 1. Instalación Node en Debian 12

El linter de javascript `eslint` necesita `node` para su funcionamiento. Aunque no es necesario para estas prácticas usaremos un gestor de versiones para la instalación de node, el gestor `nvm`. Enlaces de nod

- [Node](#)
- [nvm](#)

Los pasos a dar son:

```
# Download and install nvm:
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.2/install.sh | bash

# estarting the shell
\ . "$HOME/.nvm/nvm.sh"

# Download and install Node.js:
nvm install 22

# Verify the Node.js version:
node -v # Should print "v22.14.0".
nvm current # Should print "v22.14.0".

# Verifica versión de npm:
npm -v # Debería mostrar "10.9.2".
```

Otra posible opción sería instalar `node` (y `npm`) desde `apt`:

```
sudo apt update
sudo apt install nodejs npm
```

4.5.2 2. Instalar eslint

En la instalación de eslint puede tener algunas dificultades debido a la transición actual entre dos estilos de configuración. Los pasos a dar son:

- Iniciar el proyecto:

```
~$ mkdir mis-proyectos
~$ cd mis-proyectos/
~/mis-proyectos$ npm init -y
Wrote to /home/usuario/mis-proyectos/package.json:

{
  "name": "mis-proyectos",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}
```

- Instalar ESLint:

```
~/mis-proyectos$ npm install eslint --save-dev

added 87 packages, and audited 88 packages in 2s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
~/mis-proyectos$
```

- Ejecutar el asistente de ESLint

```
~/mis-proyectos$ npx eslint --init
You can also run this command directly using 'npm init @eslint/config@latest'.
Need to install the following packages:
```

```

@eslint/create-config@1.7.0
Ok to proceed? (y) y

> mis-proyectos@1.0.0 npx
> create-config

@eslint/create-config: v1.7.0

✓ What do you want to lint? · javascript
✓ How would you like to use ESLint? · problems
✓ What type of modules does your project use? · esm
✓ Which framework does your project use? · none
✓ Does your project use TypeScript? · no / yes
✓ Where does your code run? · browser
The config that you've selected requires the following dependencies:

eslint, @eslint/js, globals
✓ Would you like to install them now? · No / Yes
✓ Which package manager do you want to use? · npm
📦Installing...

added 1 package, changed 1 package, and audited 89 packages in 496ms

23 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
Successfully created /home/usuario/mis-proyectos/eslint.config.mjs file.
~/mis-proyectos$

```

- Probar: añadir un archivo de prueba

```

echo "let a" > index.js
npx eslint index.js

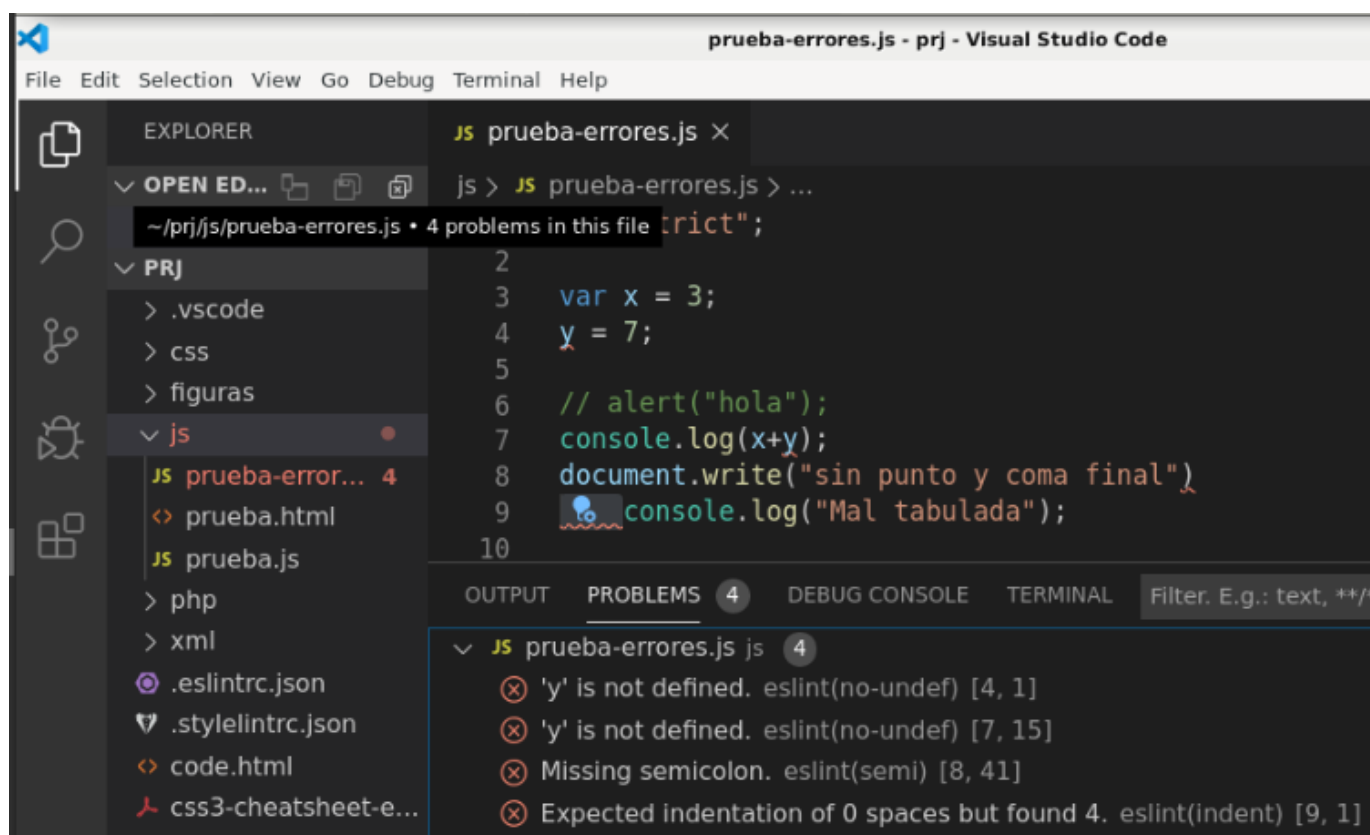
```

Más Info: <https://eslint.org/docs/latest/use/getting-started>

4.5.3 3. Uso en VS Code

Para usar el *linter* desde el editor Vscode necesita además un *plugin*. El plugin usado en clase es ESLint (de Microsoft) que integra ESLint en VS Code.

El plugin muestras los errores en el editor y en la consola de errores:



Más Info: <https://marketplace.visualstudio.com/items?itemName=dbaumer.vscod-eslint>

4.5.4 4. Reglas

Puede personalizar la configuración activando reglas adicionales, desactivado alguna de las recomendadas o modificando su configuración.

Modifique el fichero `eslint.config.mjs` añadiendo por ejemplo estas reglas

```
export default defineConfig([
  {
    files: ["**/*.js,mjs,cjs"],
    plugins: { js },
    extends: ["js/recommended"],
    // aquí va rules
    rules: {
      // Buenas prácticas
      eqeqeq: ['error', 'always'],
      'no-var': 'error',
      'prefer-const': 'error',
      'no-unused-vars': ['warn', { argsIgnorePattern: '^_', varsIgnorePattern: '^_' }],
      'no-console': 'warn',
      // Estilo
      semi: ['error', 'always'],
      indent: ['error', 2],
      'no-trailing-spaces': 'error',
    },
  },
]);
```

Puede consultar todas las reglas (y cuáles son las recomendadas) en este enlace: <https://denar90.github.io/eslint.github.io/docs/rules/>.

También es posible modificar las reglas que se comprueban y el tipo de aviso (ninguno, warning, error) en el código del propio fichero usando [configuration comments](#)

4.5.5 5. Anexos

5.1. Aclaraciones

- Si en la ruta del fichero JavaScript hay varios ficheros de configuración se usa el más cercano al fichero fuente.